CAMPUSCONNECT

A Student–Company Job and Internship Platform

A Project Report

Submitted in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

Ramkumar Harishchandra Kanojiya

Seat No: 26727

Under the esteemed guidance of

Mrs. Mona Khobare

ASST. PROFESSOR



DEPARTMENT OF COMPUTER SCIENCE

MODEL COLLEGE OF SCIENCE AND COMMERCE

(Affiliated to University of Mumbai)

KALYAN, 421306

MAHARASHTRA

2026

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL

PNR No.: _____ Roll No.: _____

1. Name of the Student: Ramkumar Harishchandra Kanojiya
2. Title of the Project: CampusConnect – A Student–Company Job and Internship Platform
3. Name of the Guide: Mrs. Mona Khobare
4. Teaching experience of the Guide: _____
5. Is this your first submission? Yes / No

Signature of the Student:
_____

Date: _____

Signature of the Guide:
_____

Date: _____

Signature of the Coordinator: _____
Date: _____



The East Kalyan Welfare Society's

MODEL COLLEGE OF SCIENCE & COMMERCE, KALYAN(E)

Accredited 'B' Grade by NAAC

(Affiliated to University of Mumbai)

This is to certify that Mr./Miss. _____

Seat No. _____ PRN No. _____ of T.Y.B.Sc (Computer Science) during Semester – V has completed the prescribed practical's of the course _____ under our supervision in Model College of Science & Commerce, Kalyan(E), affiliated to Mumbai University during the Academic Year _____.

_____ _____

TEACHER IN-CHARGE **EXTERNAL EXAMINER**

_____ _____

HEAD,
DEPARTMENT OF COMPUTER SCIENCE

PRINCIPAL

T.Y.B.Sc.CS, SEM-V

DECLARATION

I hereby declare that the project “CampusConnect – A Student–Company Job and Internship Platform” carried out at The East Kalyan Welfare Society’s Model College of Science & Commerce, Kalyan(E), has not been duplicated or submitted to any other university for the award of any degree. This project is submitted in partial fulfilment of the requirements for the award of the degree of BACHELOR OF SCIENCE (COMPUTER SCIENCE) as the Semester V project.

I further declare that the work presented in this report is my own work carried out under the guidance of the project guide.

Date: _____

Place: _____

Signature of Student: _____

Role and Responsibility Form

This project was undertaken individually. The role and responsibility of the student is summarised below.

Name	Role	Responsibility
Ramkumar Harishchandra Kanojiya	Sole Developer	Requirement analysis, database design, front-end and backend development, testing, and documentation of all modules of the CampusConnect platform.

Abstract

CampusConnect is a web-based platform that connects students seeking internships and jobs with companies looking to hire student talent. Built using PHP, MySQL and a responsive HTML/CSS front end, the system supports two user roles — students and companies — each with dedicated registration, login and dashboard experiences. Students can build a profile, browse and search job listings, apply with a cover letter, and track the status (pending, accepted or rejected) of every application they submit. Companies can post job or internship openings, review incoming applications alongside applicant resumes and profiles, and accept, reject, or reset the status of each application. The system uses PHP sessions for authentication and role-based access control to keep student and company areas separate and secure. This report documents the design, implementation and testing of the working system.

Acknowledgement

I would like to express my sincere gratitude to my project guide, Mrs. Mona Khobare, for her constant guidance, valuable suggestions and encouragement throughout the development of this project. Her feedback was instrumental in shaping both the technical implementation and the presentation of this report.

I am also thankful to the Department of Computer Science and the faculty of Model College of Science and Commerce for providing the resources and environment necessary to complete this project.

Finally, I would like to thank my family and friends for their continuous support and encouragement.

Ramkumar Harishchandra Kanojiya

Table of Contents

Sr. No.	Index	Pg. No.	Sign
1	**Original Copy of the Approved Proforma of the Project Proposal**	i	
2	**Certificate of Authenticated Work / Signature Page**	ii	
3	**Declaration**	iii	
4	**Role and Responsibility Form**	iv	
5	**Abstract**	v	
6	**Acknowledgement**	vi	
7	**List of Figures**	ix	
8	**Chapter 1: Introduction**	1	
9	1.1 Background	1	
10	1.2 Objectives	1	
11	1.3 Purpose, Scope and Applicability	1	
12	1.3.1 Purpose	1	
13	1.3.2 Scope	1	
14	1.3.3 Applicability	1	
15	1.4 Achievements	2	
16	1.5 Organisation of Report	2	
17	**Chapter 2: Survey of Technologies**	3	
18	Front End	3	
19	Back End	3	
20	Database	3	
21	Development Environment	3	
22	**Chapter 3: Requirements and Analysis**	4	
23	3.1 Problem Definition	4	
24	3.2 Requirements Specification	4	
25	3.3 Planning and Scheduling	5	
26	3.4 Software and Hardware Requirements	8	
27	3.5 Preliminary Product Description	8	
28	3.6 Conceptual Models	9	
29	**Chapter 4: System Design**	13	
30	4.1 Basic Modules	13	
31	4.2 Data Design	15	
32	4.2.1 Schema Design	15	
33	4.2.2 Data Integrity and Constraints	17	
34	4.3 Procedural Design	17	
35	4.3.1 Logic Diagrams	17	
36	4.3.2 Data Structures	19	
37	4.3.3 Algorithms Design	19	

Sr. No.	Index	Pg. No.	Sign
38	4.4 User Interface Design	20	
39	4.5 Security Issues	20	
40	4.6 Test Cases Design	20	
41	**Chapter 5: Implementation and Testing**	21	
42	5.1 Implementation Approaches	21	
43	5.2 Coding Details and Code Efficiency	21	
44	5.2.1 Code Efficiency	22	
45	5.3 Testing Approach	22	
46	5.3.1 Unit Testing	22	
47	5.3.2 Integrated Testing	22	
48	5.3.3 Beta Testing	22	
49	5.4 Modifications and Improvements	23	
50	5.5 Test Cases	24	
51	**Chapter 6: Results and Discussion**	25	
52	6.1 Test Reports	25	
53	6.2 User Documentation	25	
54	Student Workflow	26	
55	Company Workflow	39	
56	**Chapter 7: Conclusions**	49	
57	7.1 Conclusion	49	
58	7.1.1 Significance of the System	49	
59	7.2 Limitations of the System	49	
60	7.3 Future Scope of the Project	50	
61	**References**	51	
62	**Glossary**	52	
63	**Appendix A: Source Code Listings**	53	
64	**Appendix B: Installation and Setup Guide**	67	

List of Figures

Fig. No.	Caption	Pg. No.
1	Gantt Chart – Project Schedule	6
2	PERT Chart – Activity Network	7
3	Entity-Relationship Diagram	9
4	DFD Level 0 (Context Diagram)	10
5	DFD Level 1 – Job Application Process	11
6	Use Case Diagram	12
7	System Architecture (Three-Tier)	14
8	Flowchart – Student Job Application Process	18
9	Home page – hero section	26
10	Home page – Students / Companies / Opportunities section	27
11	Student Registration form (part 1)	28
12	Student Registration form (part 2)	29
13	Student Dashboard – profile and opportunities	30
14	Student Dashboard – application summary	31
15	Jobs listing – search and filters	32
16	Jobs listing – job cards with application status	33
17	Apply for Job – job summary	34
18	Apply for Job – cover letter form	35
19	Job Details page	36
20	My Applications – accepted application	37
21	My Applications – pending application	38
22	Company Dashboard	39
23	Post a Job – form validation	40
24	Post a Job – full form	41
25	Job Applications (company view) – filter tabs and accepted application	42
26	Student profile as viewed by a company	43
27	Job Applications (company view) – pending applications with Accept / Reject	44
28	My Jobs – posted job listing (1)	45
29	My Jobs – posted job listing (2)	46
30	Edit Job form	47
31	Company Profile page	48

Chapter 1: Introduction

1.1 Background

Placement and internship coordination in colleges is often handled through fragmented, manual channels such as notice boards, emails and spreadsheets. This makes it difficult for students to discover timely opportunities and for companies to reach suitable student candidates efficiently. General-purpose job portals exist, but they are not tailored to the simpler, two-role (student and company) workflow that a single college's placement activity typically needs. CampusConnect was built as a lightweight, self-contained web application to address this gap for a college setting.

1.2 Objectives

To design and develop a web-based platform where students can register, build a profile, discover and apply to job and internship postings, and track the status of their applications; and where companies can register, post job openings, and review, accept or reject student applications.

1.3 Purpose, Scope and Applicability

1.3.1 Purpose

The purpose of CampusConnect is to simplify how students discover and apply for jobs and internships, and how companies advertise openings and screen candidates, by bringing both sides of the process into a single, unified system rather than relying on scattered manual coordination.

1.3.2 Scope

The system covers registration and login for both student and company roles, profile management, job posting and browsing with search/filter, applying to jobs with a cover letter, and application status tracking and management (pending, accepted, rejected) for both students and companies. It does not cover payment processing, automated resume parsing or scoring, or interview scheduling.

1.3.3 Applicability

The system is applicable to any college or training-and-placement cell that wants a self-hosted, lightweight application to coordinate job and internship opportunities between its own students and recruiting companies, without depending on a large third-party job portal.

1.4 Achievements

A fully working two-role web application was implemented and tested end-to-end, covering registration, authentication, profile management, job posting, job search, applying with a cover letter, and the complete accept / reject / reset application-status workflow. The system is backed by a normalised MySQL database schema and presents a consistent, professional user interface across every page for both roles.

1.5 Organisation of Report

Chapter 2 surveys the technologies considered for this project. Chapter 3 covers requirements analysis, including the problem definition and software/hardware requirements. Chapter 4 presents the system design, including the database schema, procedural design and user interface design. Chapter 5 describes the implementation approach and testing carried out. Chapter 6 presents the results with screenshots of the working system. Chapter 7 concludes the report with the system's significance, limitations and future scope.

Chapter 2: Survey of Technologies

Several technology options were considered for each layer of CampusConnect before settling on the final stack.

Front End

Plain HTML and CSS were chosen over a JavaScript framework such as React or Angular. For a project of this scope — a defined set of server-rendered pages per role — a front-end framework adds build tooling and configuration overhead without a matching benefit. Plain HTML/CSS keeps every page simple to write, style and test directly in the browser.

Back End

PHP was chosen over alternatives such as Node.js/Express, Python/Django or Java Servlets. PHP integrates directly with the Apache web server and MySQL database with no additional runtime or framework to install, provides built-in session handling that is used throughout CampusConnect for authentication, and is well documented — a practical choice for a single-developer academic project on a fixed timeline.

Database

MySQL (via MariaDB) was chosen over alternatives such as PostgreSQL or SQLite. It is the database bundled by default with XAMPP, has mature support for relational schema design with foreign keys — used throughout the CampusConnect schema — and is administered easily through phpMyAdmin, which was used throughout development to design and verify the database structure.

Development Environment

XAMPP (Apache + MySQL/MariaDB + PHP) was chosen as the local development and hosting environment because it bundles all required services, avoiding separate installation and configuration of Apache, PHP and MySQL individually. Visual Studio Code was used to write and edit all PHP, HTML and CSS files.

Chapter 3: Requirements and Analysis

3.1 Problem Definition

The overall problem is that students and companies lack a simple, shared platform for discovering and applying for internship and job opportunities, and for managing those applications afterwards. This can be divided into two sub-problems: (a) students need a structured way to find opportunities, apply to them, and check the outcome of each application; and (b) companies need to publish openings and efficiently review, accept or reject applicants after viewing each applicant's profile and resume.

3.2 Requirements Specification

The functional requirements identified for each role are listed below.

Role	Requirement
Student	Register for a new account with academic details (course, college, graduation year, skills)
Student	Log in and log out securely
Student	View and edit personal profile, including uploading a resume
Student	Browse and search job/internship listings by keyword, location and job type
Student	View full details of a job listing
Student	Apply to a job with a cover letter
Student	View a history of submitted applications and their current status
Company	Register for a new account with company details
Company	Log in and log out securely
Company	View and edit company profile
Company	Post a new job or internship listing
Company	Edit, close/reopen, or delete a posted job listing
Company	View and filter applications received (all / pending / accepted / rejected)
Company	View an applicant's profile and resume
Company	Accept, reject, or reset the status of a pending application

Non-functional requirements include: passwords must never be stored in plain text; every protected page must verify the logged-in user's session and role before granting access; and the interface must present a consistent, professional design across both roles and remain usable on smaller screen widths.

3.3 Planning and Scheduling

The project was developed iteratively, module by module, from early August to mid-September 2026. The table below shows the approximate period of each phase.

Phase	Activity	Approximate Period
1	Requirement analysis and database schema design	Early August 2026
2	Authentication, registration and dashboards	Mid August 2026
3	Job posting, browsing and application workflow	Late August 2026
4	Company-side application review and status management	Early September 2026
5	User interface redesign across all pages	Early–mid September 2026
6	Testing, bug fixing and documentation	Mid September 2026

Planning identifies the tasks that must be carried out to complete the project, while scheduling decides when each task is carried out. The schedule of CampusConnect is presented on the following pages as a Gantt chart (Figure 1) and a PERT chart (Figure 2).

The Gantt chart shows the six phases of the project against a timeline that runs from early August to mid-September 2026, divided into Early, Mid and Late parts of each month. Each bar covers the approximate period in which the phase was carried out. Phases 1, 2 and 3 followed one another through August, Phases 4 and 5 overlapped in early September, and Phase 6 — testing, bug fixing and documentation — completed the project in mid-September.

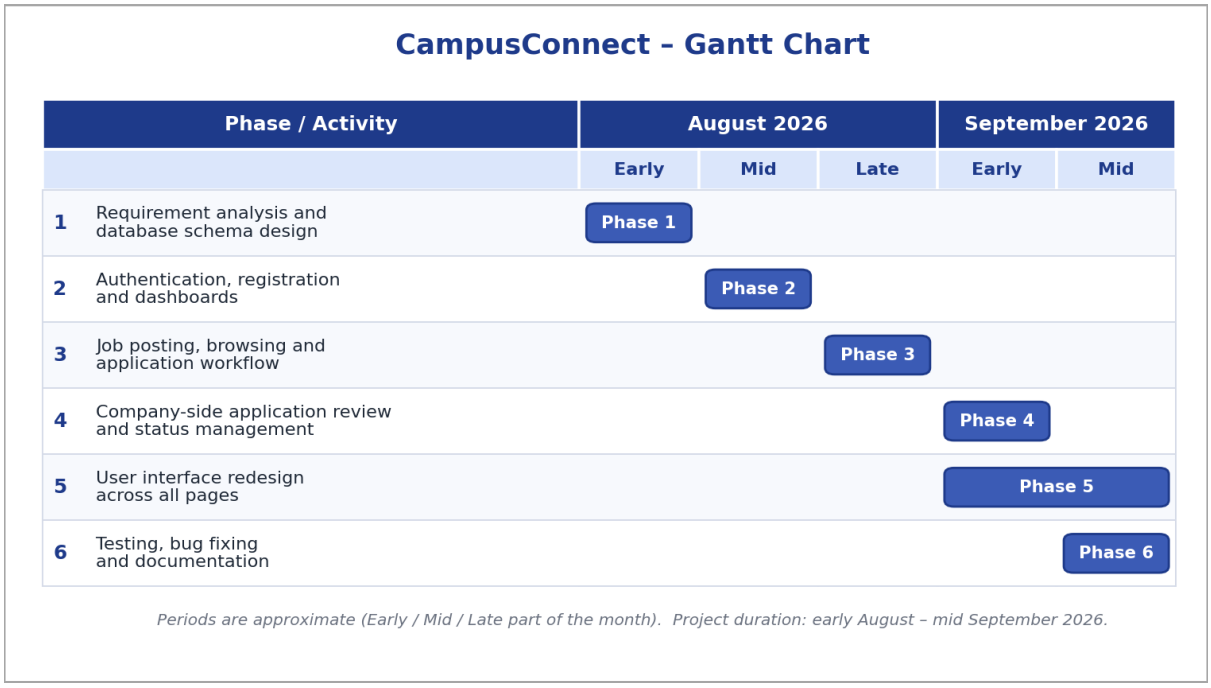


Figure 1: *Gantt Chart – Project Schedule*

The PERT chart shows the project as a network of numbered events (circles) joined by activities (arrows labelled A to F). Activities A, B and C follow one another. After activity C, the company-side review activity D and the user interface redesign E begin; D is followed by testing and documentation F, while E runs alongside them. Each event shows its earliest time (TE) and latest time (TL), measured in periods of Early, Mid or Late month. Because TE and TL are equal at every event, there is no slack, both paths are critical, and the project length is five periods, from early August to mid-September 2026.

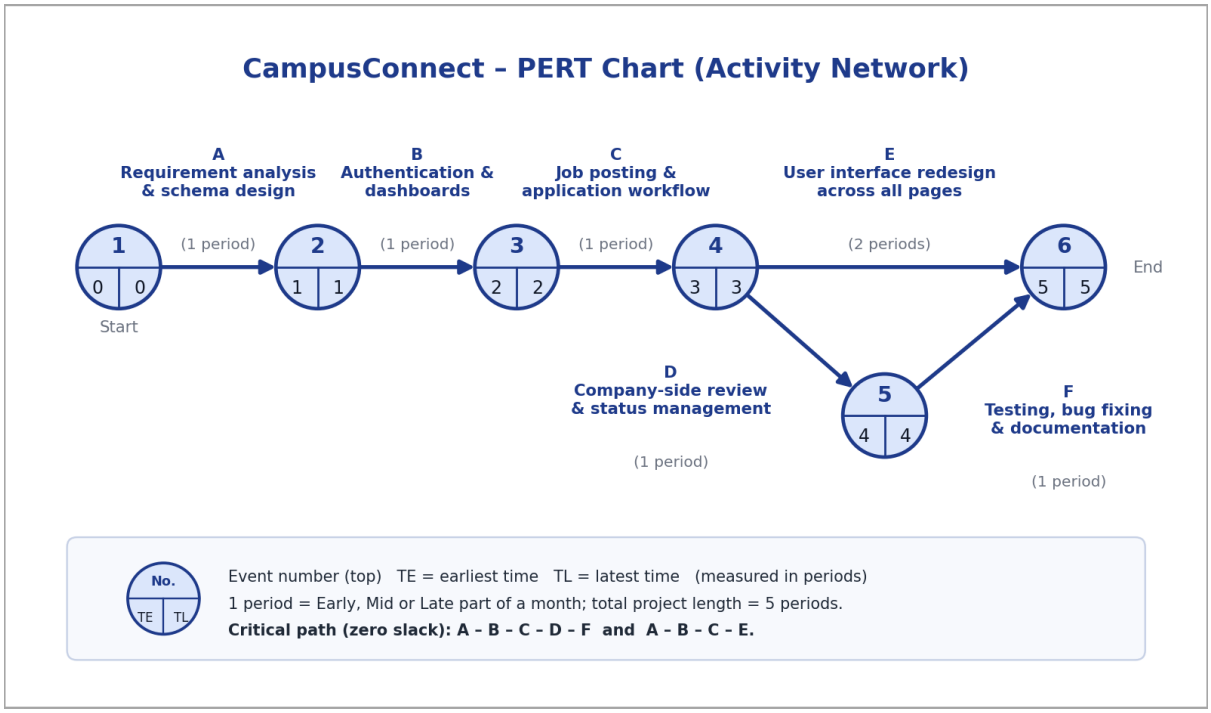


Figure 2: *PERT Chart – Activity Network*

3.4 Software and Hardware Requirements

Software Requirements

- Operating System: Windows 10/11 (or any OS supporting XAMPP)
- Server Stack: XAMPP — Apache, PHP 8.2.12, MySQL/MariaDB
- Code Editor: Visual Studio Code
- Database Administration: phpMyAdmin (bundled with XAMPP)
- Web Browser: Google Chrome or any modern browser

Hardware Requirements (minimum)

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 500 MB free space for the project and XAMPP installation
- Standard keyboard, mouse and monitor

3.5 Preliminary Product Description

CampusConnect is a two-role web application. A student can create an account, maintain a profile describing their course, college, graduation year, skills and resume, browse and search active job listings posted by companies, and apply to any listing with a cover letter. Every application a student submits can later be tracked from a dedicated My Applications page, which shows its current status. A company can create an account, maintain a company profile, post job or internship listings, and manage every application received against its own listings — viewing the applicant's profile and resume before accepting, rejecting, or resetting an application back to pending.

3.6 Conceptual Models

The conceptual structure of CampusConnect is represented in the following pages through an Entity-Relationship Diagram, a Data Flow Diagram (Level 0 and Level 1), and a Use Case Diagram. Each diagram is presented on a separate page together with a short explanation.

The Entity-Relationship Diagram presents the logical structure of the campusconnect_db database. It contains five entities — Users, Students, Companies, Jobs and Applications — together with their attributes and keys. Each student profile is linked one-to-one with a user account, each company can post many jobs, and the Applications entity links students to jobs, so one student can apply to many jobs and one job can receive many applications. The Users and Companies entities are matched by email address at the application level rather than through a database foreign key.

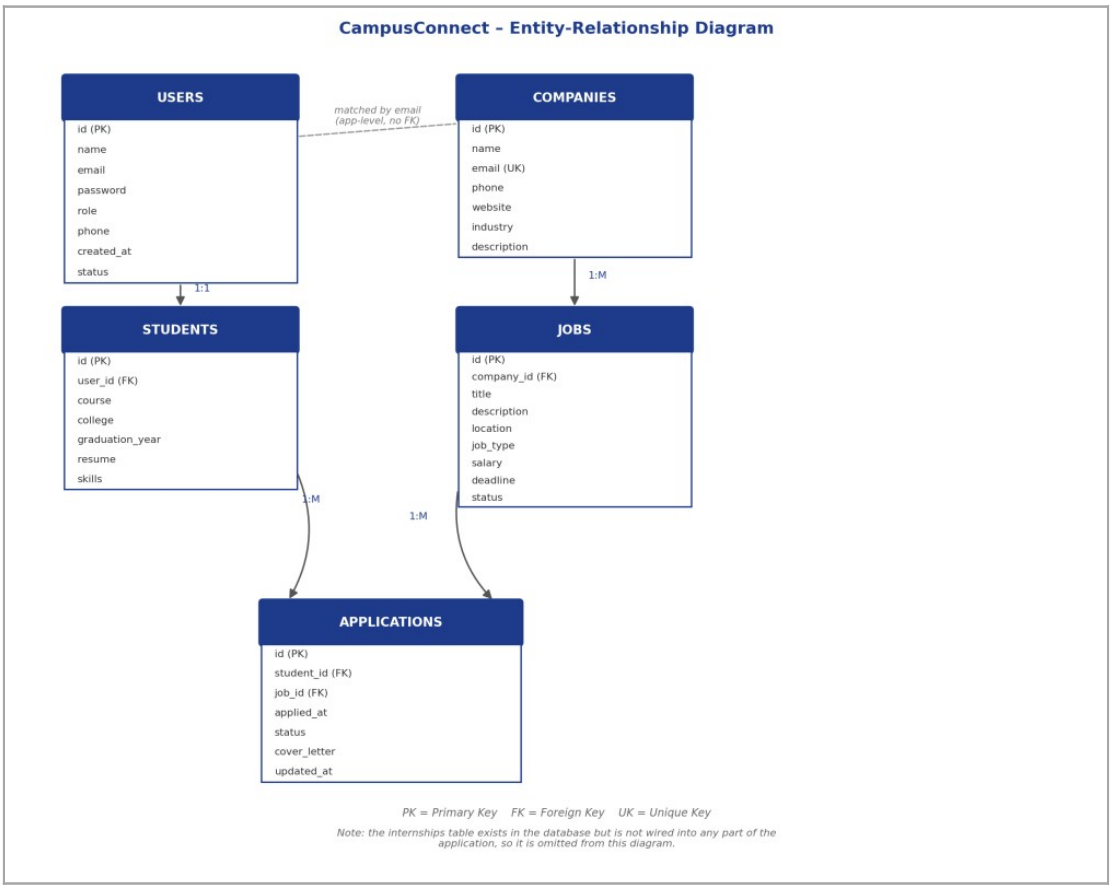


Figure 3: *Entity-Relationship Diagram*

The Level 0 Data Flow Diagram, also called the context diagram, shows CampusConnect as a single process (0.0) that interacts with its two external entities. The Student sends registration, login, job search and application data to the system and receives job listings and application status updates in return. The Company sends registration, login, job posting and accept / reject data, and receives its posted jobs, the applications received and the applicants' resumes.

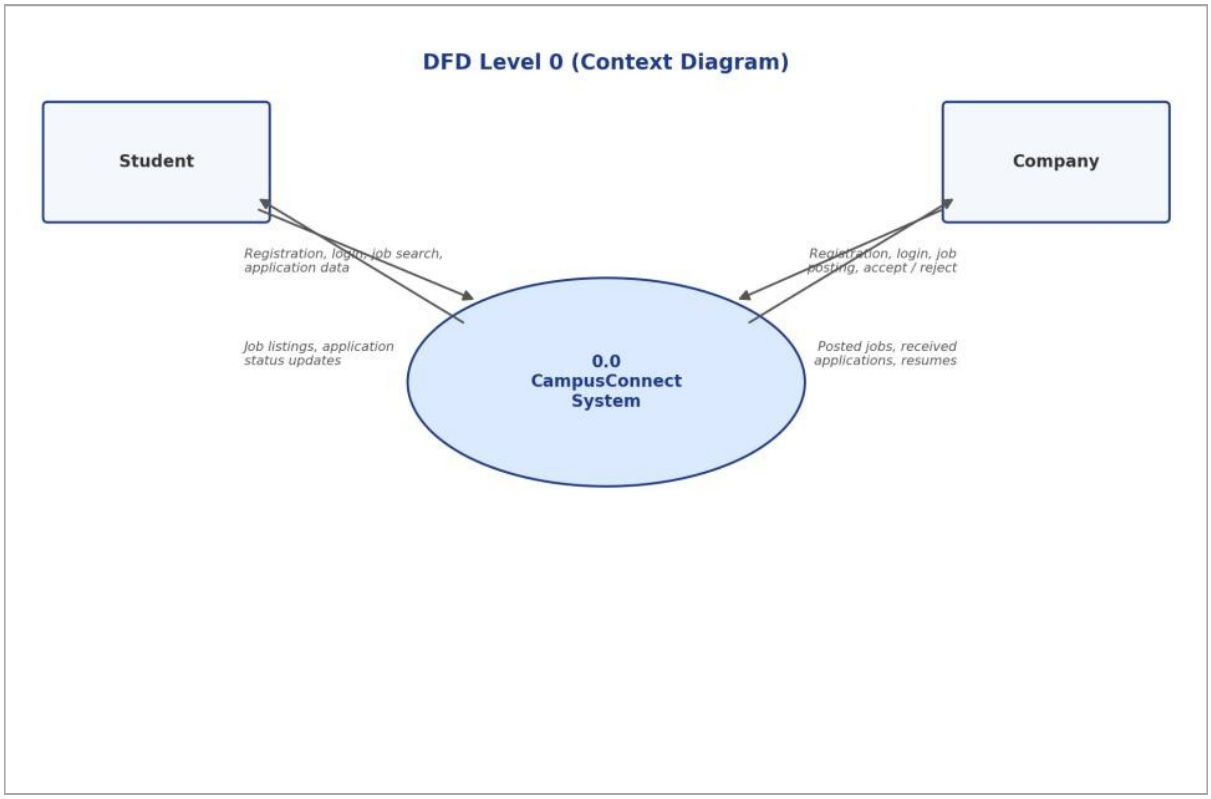


Figure 4: *DFD Level 0 (Context Diagram)*

The Level 1 Data Flow Diagram expands the job application process into three processes: 1.0 Browse & View Jobs, 2.0 Submit Application and 3.0 Review & Update Status. These processes work with three data stores — D1: jobs, D2: applications and D3: students. The student's request returns the job list, the cover letter is submitted through process 2.0 and written to the applications store, and the company's accept / reject action updates the stored status, reads the student's profile and returns the status result to the company.

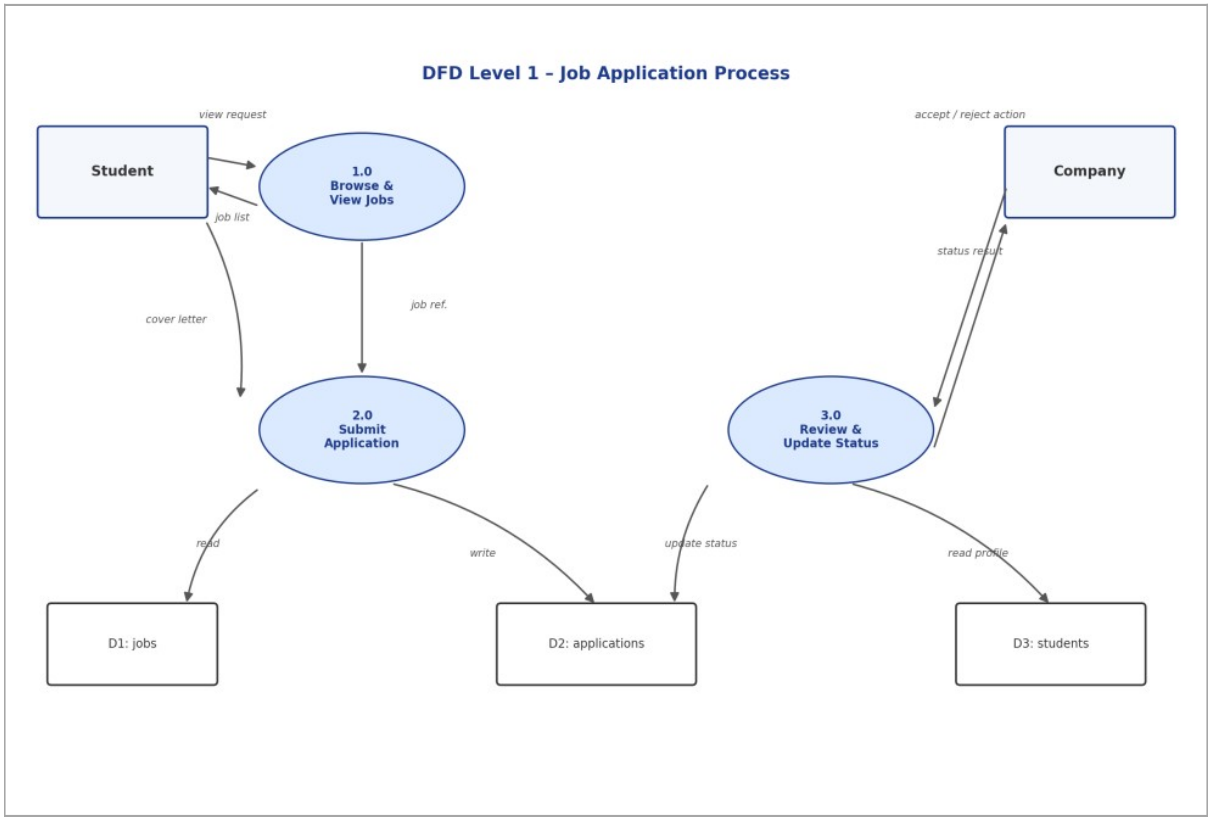


Figure 5: *DFD Level 1 – Job Application Process*

The Use Case Diagram identifies the two actors of the system and the functions available to each. The Student can register and log in, view and edit the profile, browse jobs, view job details, apply for a job and track applications. The Company can register and log in, manage the company profile, post a job, manage posted jobs, view applications, accept or reject an application and view a student's profile.

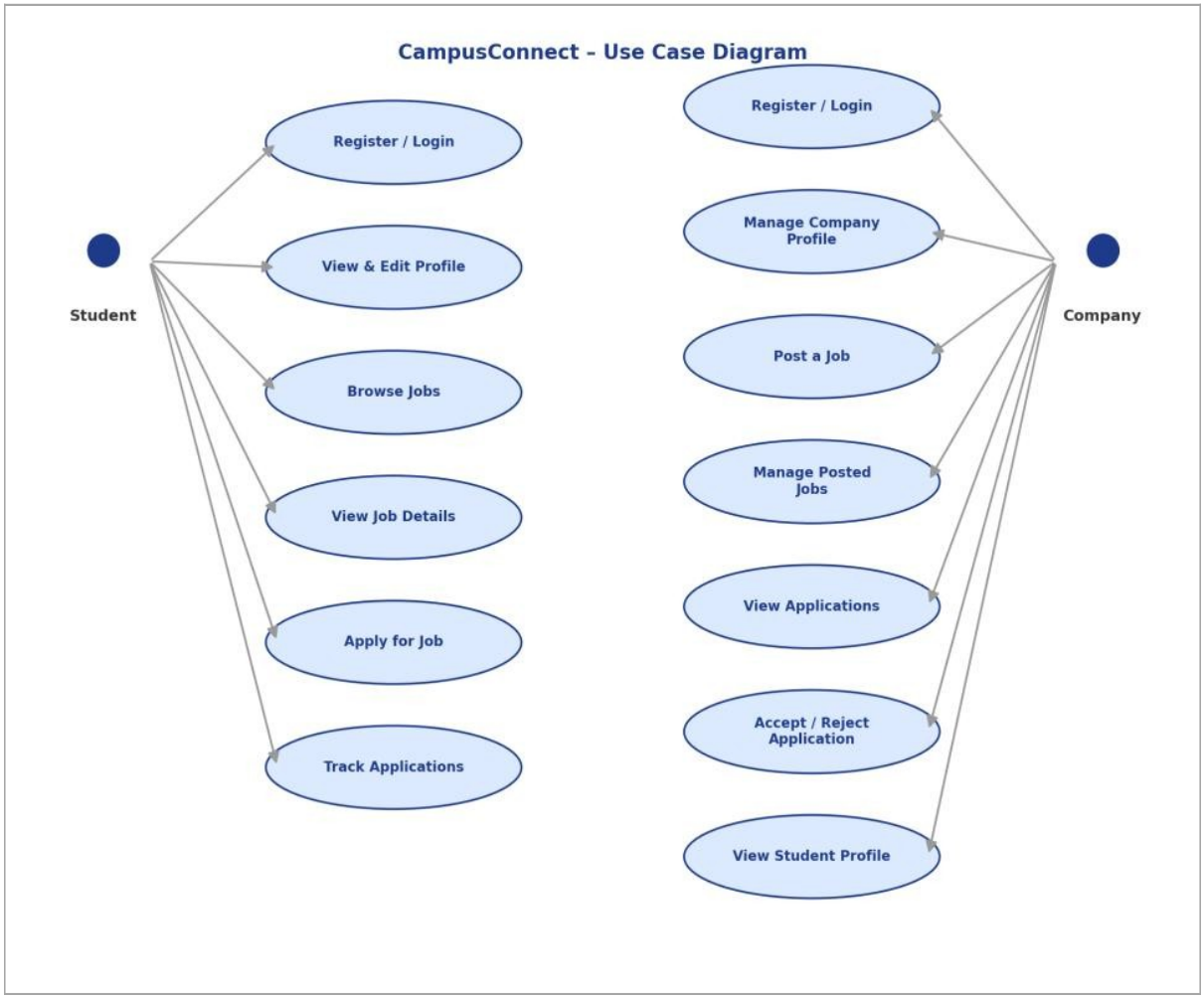


Figure 6: *Use Case Diagram*

Chapter 4: System Design

4.1 Basic Modules

CampusConnect is organised into the following modules:

- **Authentication Module** — registration (student and company), login, logout, and session-based role checking on every protected page.
- **Student Module** — profile view/edit, browse and search jobs, view job details, apply with a cover letter, view application history and status.
- **Company Module** — company profile, post/edit/close/delete a job, view and filter received applications, view an applicant's profile and resume, accept/reject/reset an application.

The overall system architecture, showing how these modules interact with the web server and database, is presented on the following page.

CampusConnect follows a three-tier architecture. The client tier is the web browser, which sends HTTP requests (GET / POST) to the application tier, where the Apache web server runs PHP 8.2 and holds the business logic. The application tier sends SQL queries, written as prepared statements, to the data tier — the MySQL / MariaDB database campusconnect_db — and receives the result set, which is returned to the browser as an HTML / CSS response. The whole stack runs locally on the XAMPP server package.

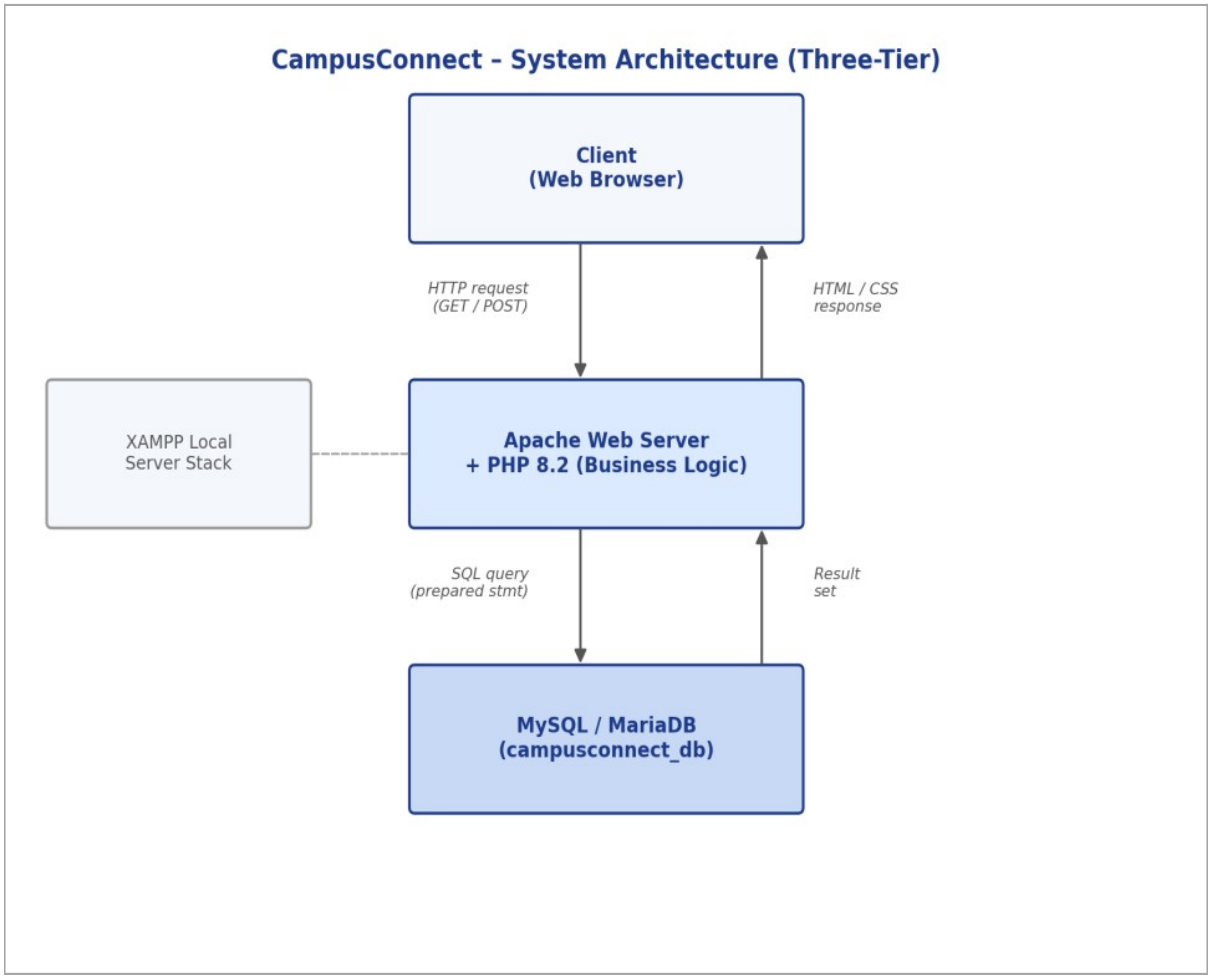


Figure 7: *System Architecture (Three-Tier)*

4.2 Data Design

4.2.1 Schema Design

The database campusconnect_db contains five tables that are actively used by the application: users, students, companies, jobs and applications. The exact structure of each, taken from phpMyAdmin, is given below.

Table: users

Column	Type	Key / Notes	Nullability
id	int(11)	PK, AUTO_INCREMENT	Not Null
name	varchar	—	Not Null
email	varchar	—	Not Null
password	varchar	— (bcrypt hash)	Not Null
role	varchar	— (student / company)	Not Null
phone	varchar	—	Not Null
created_at	timestamp	—	Not Null
status	varchar	— (active)	Not Null

Table: students

Column	Type	Key / Notes	Nullability
id	int(11)	PK, AUTO_INCREMENT	Not Null
user_id	int(11)	FK → users.id (fk_students_user)	Not Null
course	varchar(100)	—	Not Null
college	varchar(150)	—	Not Null
graduation_year	year(4)	—	Not Null
resume	varchar(255)	—	Not Null
skills	text	—	Not Null

Table: companies

Column	Type	Key / Notes	Nullability
id	int(11)	PK, AUTO_INCREMENT	Not Null
name	varchar(150)	—	Not Null
email	varchar(150)	UNIQUE	Not Null
phone	varchar(15)	—	Not Null
website	varchar(255)	—	Not Null
industry	varchar(100)	—	Not Null
description	text	—	Not Null

Table: jobs

Column	Type	Key / Notes	Nullability
id	int(11)	PK, AUTO_INCREMENT	Not Null
company_id	int(11)	FK → companies.id (fk_jobs_company)	Not Null
title	varchar(150)	—	Not Null
description	text	—	Not Null
location	varchar(100)	—	Not Null
job_type	varchar(50)	—	Not Null
salary	decimal(10,2)	—	Not Null
deadline	date	—	Not Null
status	varchar(20)	— (active / inactive)	Not Null

Table: applications

Column	Type	Key / Notes	Nullability
id	int(11)	PK, AUTO_INCREMENT	Not Null
student_id	int(11)	FK → students.id (fk_applications_student)	Not Null
job_id	int(11)	FK → jobs.id (fk_applications_job)	Not Null
applied_at	timestamp	default CURRENT_TIMESTAMP	Not Null
status	varchar(30)	— (pending / accepted / rejected)	Not Null
cover_letter	text	—	Not Null
updated_at	timestamp	default NULL	Nullable

Note: the database also contains a sixth table, internships (id, student_id, company_id, start_date, end_date, stipend, status, certificate), which was created during planning but is not read from or written to by any part of the current application (it holds zero rows). It has therefore been excluded from the design above and is discussed instead under Future Scope in Chapter 7.

4.2.2 Data Integrity and Constraints

- id columns in every table are auto-incrementing primary keys.
- students.user_id is a foreign key referencing users.id (fk_students_user), ensuring every student profile belongs to a real user account.
- jobs.company_id is a foreign key referencing companies.id (fk_jobs_company).
- applications.student_id and applications.job_id are foreign keys (fk_applications_student, fk_applications_job) ensuring every application references a real student and a real job.
- companies.email carries a UNIQUE index, preventing duplicate company email addresses at the companies table level.
- Every column across the schema is NOT NULL except applications.updated_at, which defaults to NULL until an application's status is first changed.
- Passwords are never stored in plain text — they are hashed with PHP's password_hash() before being inserted, and verified with password_verify() at login.
- The relationship between users and companies is not enforced by a database foreign key; it is matched at the application level by comparing users.email = companies.email in the PHP code.

4.3 Procedural Design

4.3.1 Logic Diagrams

The flowchart on the following page traces the logic of a student applying for a job, including both validation branches handled by the system.

The flowchart shows the logic followed when a student applies for a job. After opening a job, the system first checks whether the student has already applied; if so, the current status is shown and re-application is prevented. Otherwise the student fills in the cover letter, which is validated for a minimum of 50 characters. A valid cover letter is inserted into the applications table with the status pending and the student is redirected to the My Applications page, while an invalid one returns a validation error.

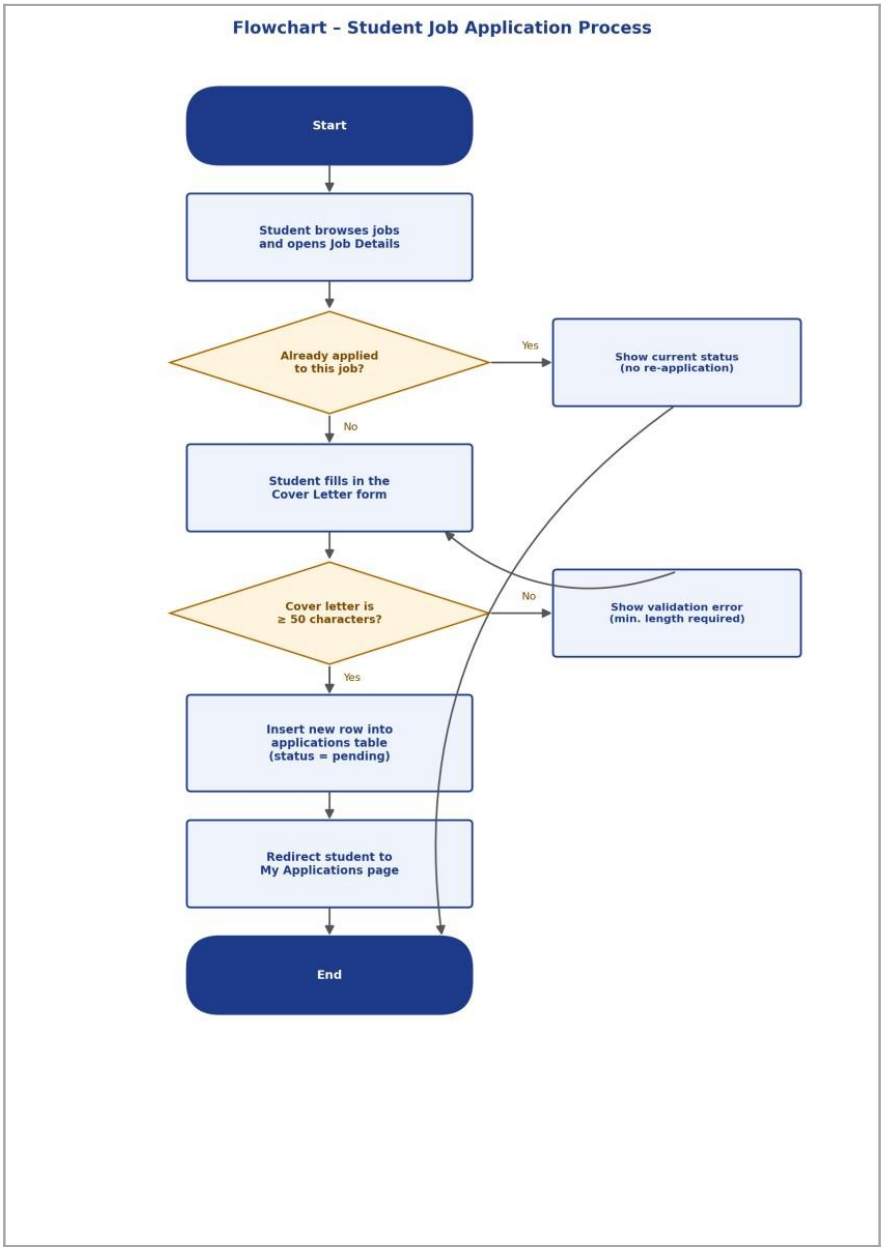


Figure 8: *Flowchart – Student Job Application Process*

4.3.2 Data Structures

The application relies mainly on three PHP data structures: associative arrays returned by mysqli's fetch_assoc() to hold a single database row (e.g. a job, a student profile, an application); indexed/associative arrays built from fetch_all() equivalents (via while loops) to hold a full result set such as a list of jobs or applications; and PHP's built-in $_SESSION superglobal array, which stores user_id, name and role for the duration of a logged-in user's browser session and is read on every protected page to enforce access control.

4.3.3 Algorithms Design

Login algorithm:

1. Read email and password from the submitted form.
2. Look up the user record in the users table by email.
3. If no matching record: show "Invalid email or password".
4. If the account status is not "active": show "Your account is not active".
5. Else, verify the submitted password against the stored hash using password_verify().
6. If verified: store user_id, name and role in $_SESSION, then redirect to the student or company dashboard based on role.
7. If not verified: show "Invalid email or password".

Apply-for-job algorithm:

1. Confirm the logged-in user is a student; otherwise redirect to login.
2. Load the job by job_id; if not found or inactive, show an error.
3. Check whether an application already exists for this (student_id, job_id) pair.
4. If it exists: show the current status instead of the form.
5. Else: display the cover-letter form.
6. On submit, validate the cover letter is at least 50 characters.
7. If valid: insert a new row into applications with status = pending.
8. Redirect the student to the My Applications page.

Accept / reject application algorithm:

1. Confirm the logged-in user is a company; otherwise redirect to login.
2. Read application_id and the requested status from the request.
3. Update the applications row ONLY IF its job_id belongs to a job posted by the logged-in company (ownership check via a join through the jobs table), preventing one company from changing another company's applications.
4. Set status to the requested value (accepted / rejected / pending) and update updated_at.
5. Redirect back to the Job Applications page.

4.4 User Interface Design

Every page in CampusConnect follows a single, consistent design system built on a navy-blue (#1e3a8a) and light-background (#f4f7fb) colour theme. Forms and content sections are presented as white, rounded cards with soft drop shadows; multi-field forms use a small icon badge and heading at the top and group related fields under short, uppercase section labels (for example “ACCOUNT DETAILS”, “ACADEMIC INFO”). Application and job statuses are shown as colour-coded pill badges — amber for pending, green for accepted/active, red for rejected — so status is recognisable at a glance across every page that shows it. The layout is responsive, adjusting to a single column on narrower screens. The complete set of interface screenshots, covering every page for both roles, is presented in Chapter 6, Section 6.2.

4.5 Security Issues

- Passwords are hashed using PHP’s password_hash() (bcrypt) and verified with password_verify() — plain-text passwords are never stored or compared.
- Every database query uses mysqli prepared statements with bound parameters (bind_param), which prevents SQL injection from user-supplied input.
- Every protected page begins by checking $_SESSION["user_id"] and $_SESSION["role"], redirecting unauthenticated users or users of the wrong role to login.php.
- Server-side ownership checks (e.g. matching a job’s company_id to the logged-in company, or an application’s student_id to the logged-in student) ensure a user can only act on records that belong to them, rather than relying only on hiding UI elements.
- User-supplied output (names, job titles, cover letters, etc.) is escaped with htmlspecialchars() before being rendered into HTML, mitigating cross-site scripting (XSS).
- Resume uploads are restricted to PDF, DOC and DOCX files, limited to 5 MB, and stored under a randomly generated filename (uniqid()) rather than the original uploaded filename.

4.6 Test Cases Design

Test cases were designed to cover, for every major feature, at least one valid/expected path and at least one invalid or edge-case path, as well as cross-role access attempts (e.g. a student trying to open a company-only page). The full set of test cases and their results is presented in Chapter 5, Section 5.5.

Chapter 5: Implementation and Testing

5.1 Implementation Approaches

CampusConnect was implemented incrementally, module by module: first authentication and the two registration flows (student and company), then the two dashboards, then job posting and browsing, then the apply and application-tracking flow, and finally the company-side application review and status-management flow. Each module was manually tested in the browser as it was completed, using a small set of test student and company accounts, before moving on to the next module.

5.2 Coding Details and Code Efficiency

A few representative code excerpts are given below to illustrate the key mechanisms used throughout the project; the complete source code for representative files is included in Appendix A.

Session-based role check, repeated at the top of every protected page:

```
session_start();

if (!isset($_SESSION["user_id"]) || $_SESSION["role"] !== "company") {
    header("Location: ../login.php");
    exit;
}
```

Password hashing at registration:

```
$hashed_password = password_hash($password, PASSWORD_DEFAULT);

$stmt->bind_param(
    "ssssss",
    $name, $email, $hashed_password, $role, $phone, $status
);
```

Password verification at login:

```
if ($result->num_rows === 1) {
    $user = $result->fetch_assoc();

    if ($user["status"] !== "active") {
        $message = "Your account is not active.";
    } elseif (password_verify($password, $user["password"])) {
        $_SESSION["user_id"] = $user["id"];
        $_SESSION["name"] = $user["name"];
        $_SESSION["role"] = $user["role"];
        // ... redirect based on role
    } else {
        $message = "Invalid email or password.";
    }
}
```

Prepared-statement lookup joining users and companies by email:

```
$company_stmt = $conn->prepare(
    "SELECT companies.id, companies.name, companies.email
     FROM users
     INNER JOIN companies ON users.email = companies.email
     WHERE users.id = ?"
);
$company_stmt->bind_param("i", $user_id);
$company_stmt->execute();
$company = $company_stmt->get_result()->fetch_assoc();
```

5.2.1 Code Efficiency

Prepared statements are used for every query that includes user-supplied values, which both prevents SQL injection and allows the database to reuse the same query plan across repeated calls to structurally identical statements (e.g. the same application-status update running for many different applications). Queries select only the columns actually needed by the page rather than using SELECT *, and MySQL automatically indexes primary-key and foreign-key columns, keeping the join-heavy queries used throughout the project — such as the applications listing that joins jobs, students and users — efficient at the scale of data a single college would generate.

5.3 Testing Approach

5.3.1 Unit Testing

Individual features were tested in isolation — for example, the registration form's password-confirmation check, the login form's credential check, and the apply form's 50-character minimum cover-letter length — with both valid and invalid input.

5.3.2 Integrated Testing

Multi-step flows spanning several pages and both roles were tested end-to-end. For example: a company posts a job, a student applies to it, the company accepts the application, and the student's My Applications page is checked to confirm the status updated correctly from pending to accepted.

5.3.3 Beta Testing

The completed system was exercised informally using multiple test student accounts and one test company account to simulate realistic day-to-day use — posting several jobs, submitting several applications, and moving them through every status — which surfaced issues such as the job-details bug described in Section 5.4.

5.4 Modifications and Improvements

- Fixed a bug where the student-side Job Details page (job_details.php) returned “Job not found or no longer available” when opened from the My Applications page. The query required jobs.status = 'active', which failed to correctly handle a job a student had already applied to; the query was corrected so a previously applied-to job now opens correctly.
- Redesigned the entire user interface, part-way through development, from a plain, unstyled layout into a consistent, professional card-based design system (described in Section 4.4) applied uniformly across every page for both roles.
- Simplified an initially-planned email-based password reset (which would have required configuring an external SMTP/Mailtrap service) into a direct in-app reset flow — the user enters their registered email, and if it matches an account they are taken straight to a page to set a new password — to avoid depending on external email infrastructure for a local academic project.

5.5 Test Cases

The test cases designed in Section 4.6 and their results are listed below.

ID	Test Case	Expected Result	Result
TC01	Student registration with valid, unique details	Account created; redirected to login	Pass
TC02	Student registration with mismatched password/confirm password	“Passwords do not match” shown; account not created	Pass
TC03	Student login with correct credentials	Redirected to Student Dashboard	Pass
TC04	Student login with incorrect password	“Invalid email or password” shown	Pass
TC05	Company registration with valid details	Account and company profile created	Pass
TC06	Company posts a job with all required fields	Job saved; appears on My Jobs and student Jobs listing	Pass
TC07	Company posts a job with a required field left blank	Browser validation blocks submission	Pass
TC08	Student applies to a job with a valid cover letter	Application saved with status “pending”	Pass
TC09	Student re-opens a job already applied to	Current status shown instead of the apply form	Pass
TC10	Company accepts a pending application	Status updates to “accepted” and reflects on the student’s My Applications page	Pass
TC11	Company rejects a pending application	Status updates to “rejected”	Pass
TC12	Company resets an accepted/rejected application	Status returns to “pending”	Pass
TC13	Unauthenticated user opens student/dashboard.php directly	Redirected to login.php	Pass
TC14	Logged-in student opens a company-only page directly	Redirected to login.php	Pass
TC15	Company opens an applicant’s profile from Job Applications	Applicant’s academic info, skills and resume link shown	Pass

Chapter 6: Results and Discussion

6.1 Test Reports

All test cases listed in Section 5.5 produced their expected result. The most important evidence of this is that the same application record can be observed moving correctly through its full lifecycle — shown as “pending” in Figure 21, and as “accepted” for a different application in Figures 20 and 25 — confirming the accept / reject / reset workflow between the student and company sides of the system functions correctly end-to-end, and that a status change made from the company side is correctly reflected back on the student side.

6.2 User Documentation

This section walks through the working system as experienced by each role, with a screenshot of every major page. It also serves as a basic user guide to the completed application. Each screenshot is presented on its own page with a short explanation, beginning with the screens used by a student and followed by the screens used by a company.

Student Workflow

A new visitor lands on the CampusConnect home page, which introduces the platform and links to registration and login. The page shows the platform name in the navigation bar together with the Home, About and Login links, a welcome banner, and the Get Started and Learn More buttons from which a visitor can proceed.

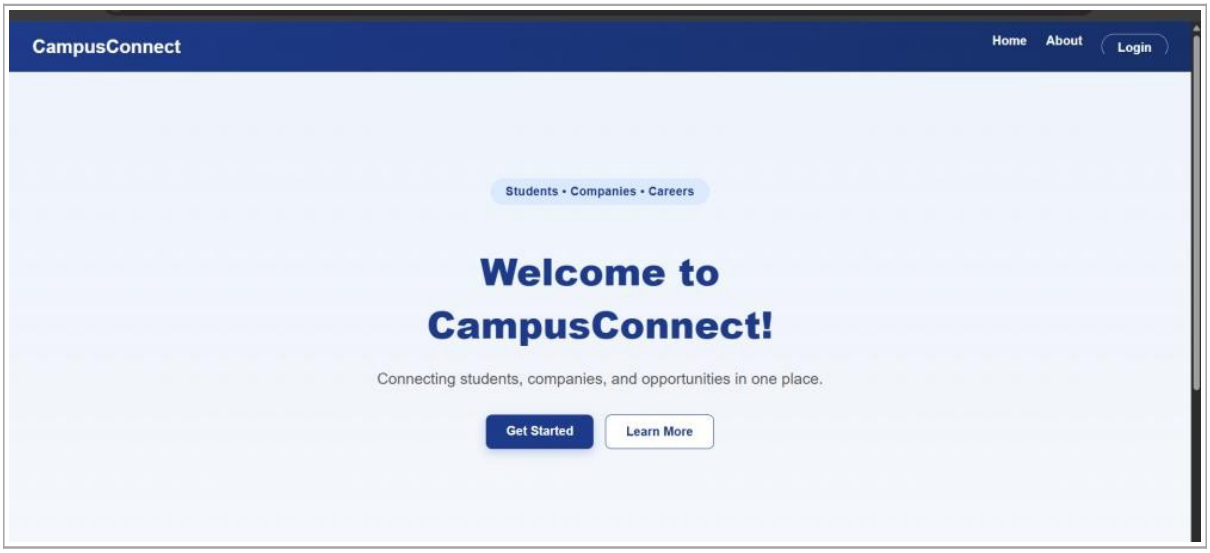


Figure 9: *Home page – hero section*

Below the welcome banner, the home page presents three highlight cards — Students, Companies and Opportunities. They summarise what the platform offers: students can create a profile and discover opportunities, companies can find talented students and connect with the right candidates, and both sides can find internships, jobs and career opportunities in one place.

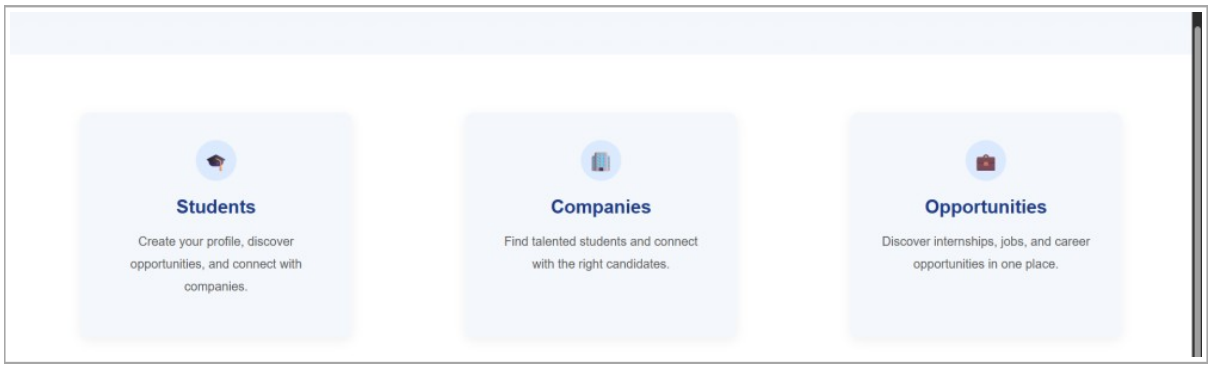


Figure 10: *Home page – Students / Companies / Opportunities section*

A student registers with their account and academic details. The first part of the Student Registration form collects the account details — full name, email address, password and confirm password, each password field having a Show option — and begins the academic information section with the course and college fields.

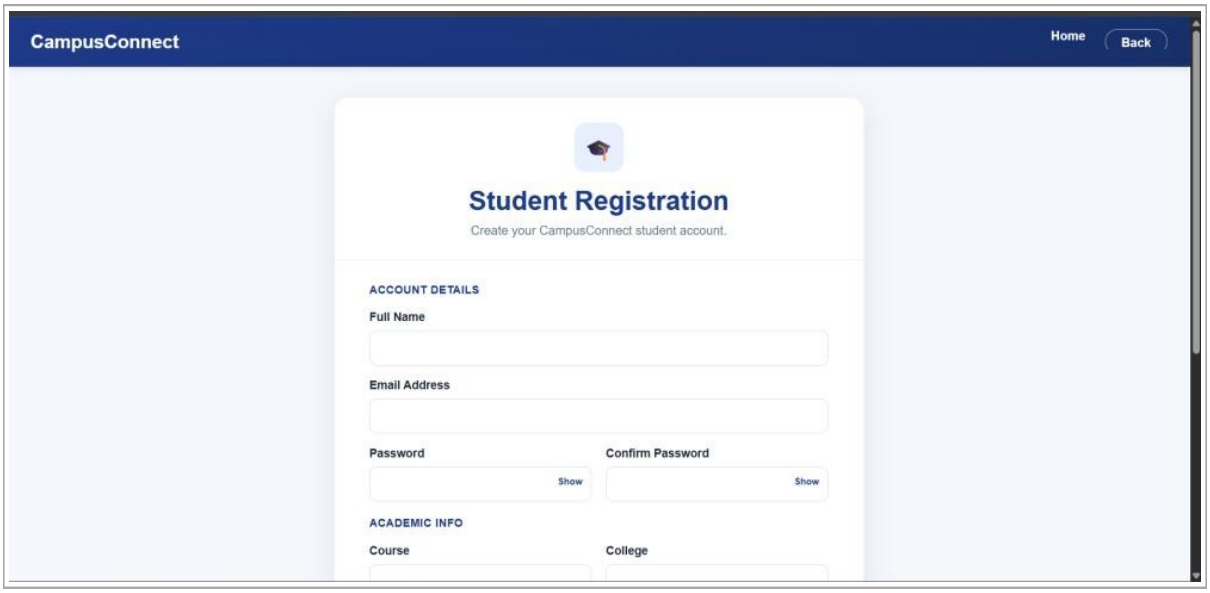


Figure 11: *Student Registration form (part 1)*

The second part of the Student Registration form completes the academic information with the graduation year and skills fields. The student submits the form with the Create Account button, and an existing user can use the Log in link at the bottom of the card.

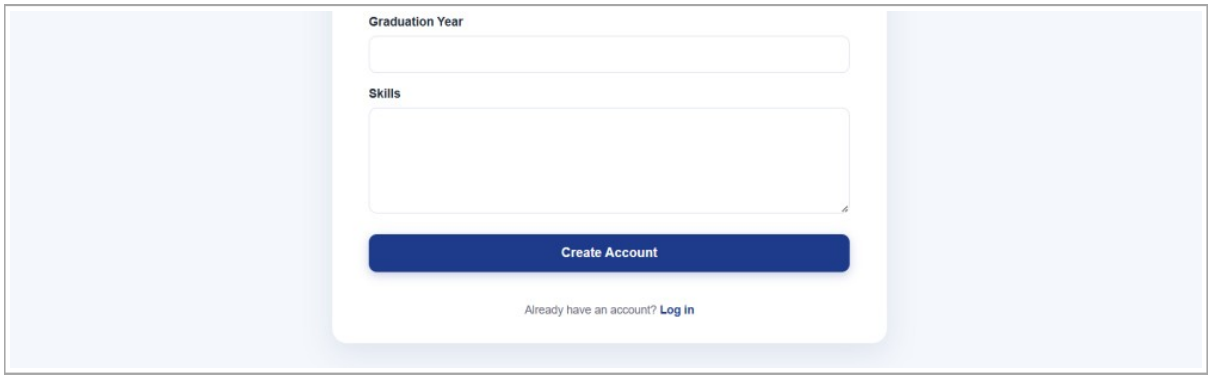


Figure 12: *Student Registration form (part 2)*

After logging in, the student sees a dashboard summarising their profile and application activity. The My Profile card displays the course, college, graduation year and skills, with View Full Profile and Edit Profile buttons, while the Opportunities card leads to the job listings through View Jobs and to the student's submissions through My Applications.

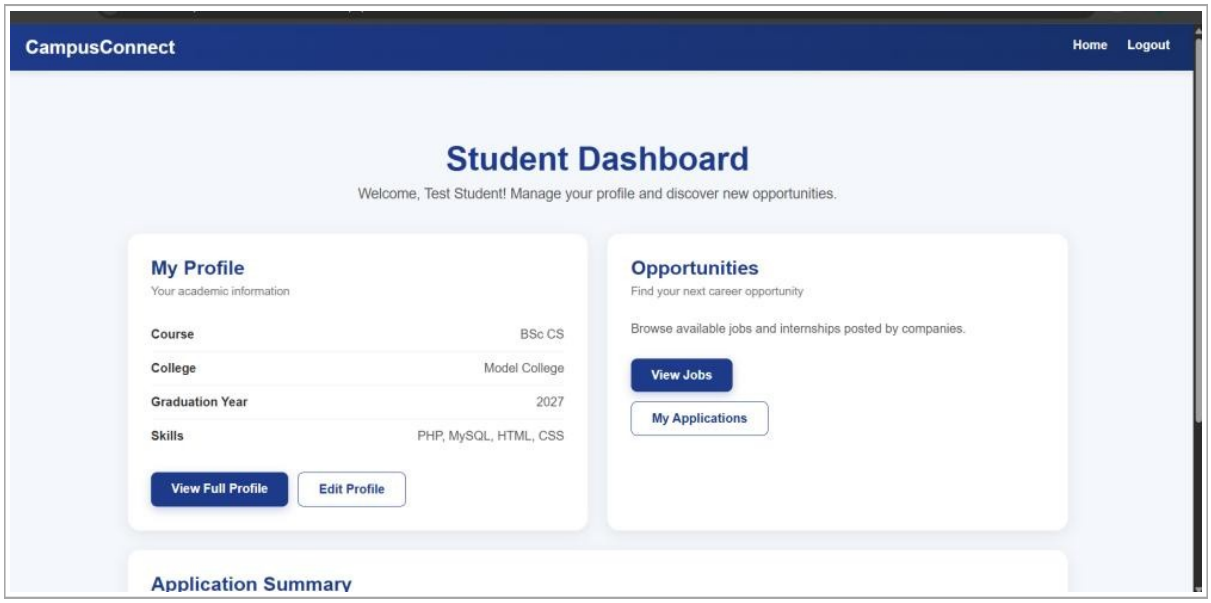


Figure 13: *Student Dashboard – profile and opportunities*

The lower part of the Student Dashboard contains the Application Summary, which gives the student a quick count of their applications — total, pending, accepted and rejected — and a View My Applications button that leads to the full list of submitted applications.

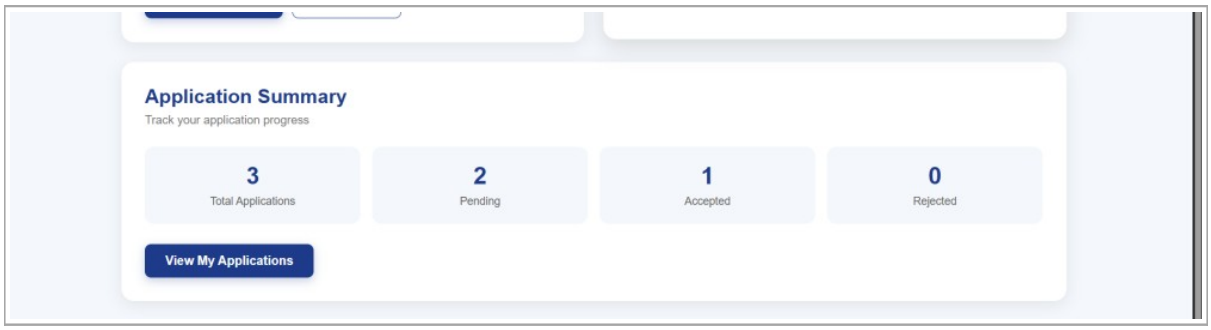


Figure 14: *Student Dashboard – application summary*

From the dashboard, the student can browse and search all active job listings. The Available Jobs page provides a keyword search box, a location box and a job-type drop-down, with Search and Clear Filters buttons to apply or reset the filters.

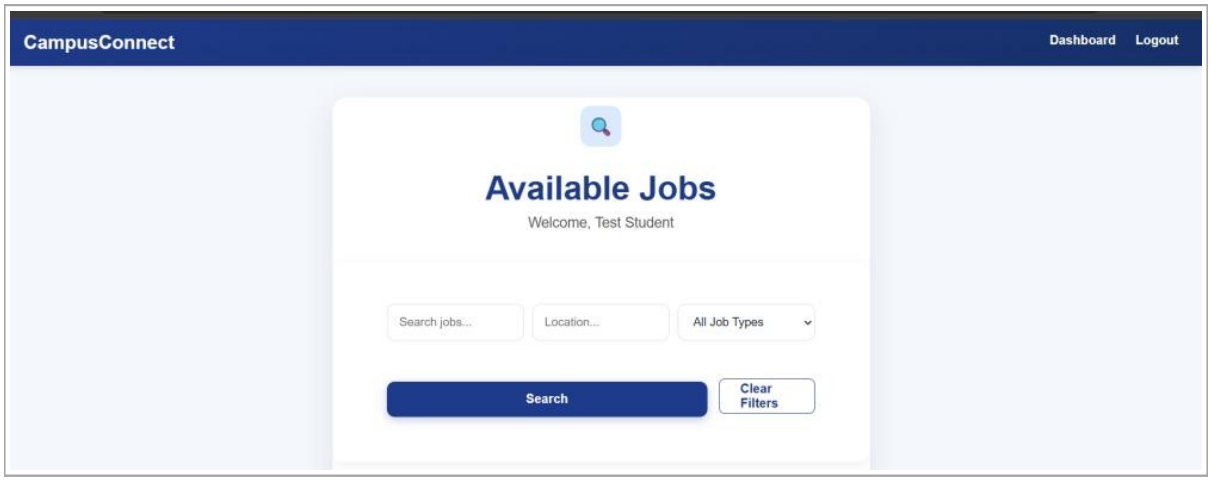


Figure 15: *Jobs listing – search and filters*

Each matching job is shown as a card with the job title, a job-type tag, the company name, location, salary, application deadline and a short description. The View Details button opens the job, and a colour-coded badge beside it — such as Accepted or Already Applied — tells the student the state of any application already made for that job.

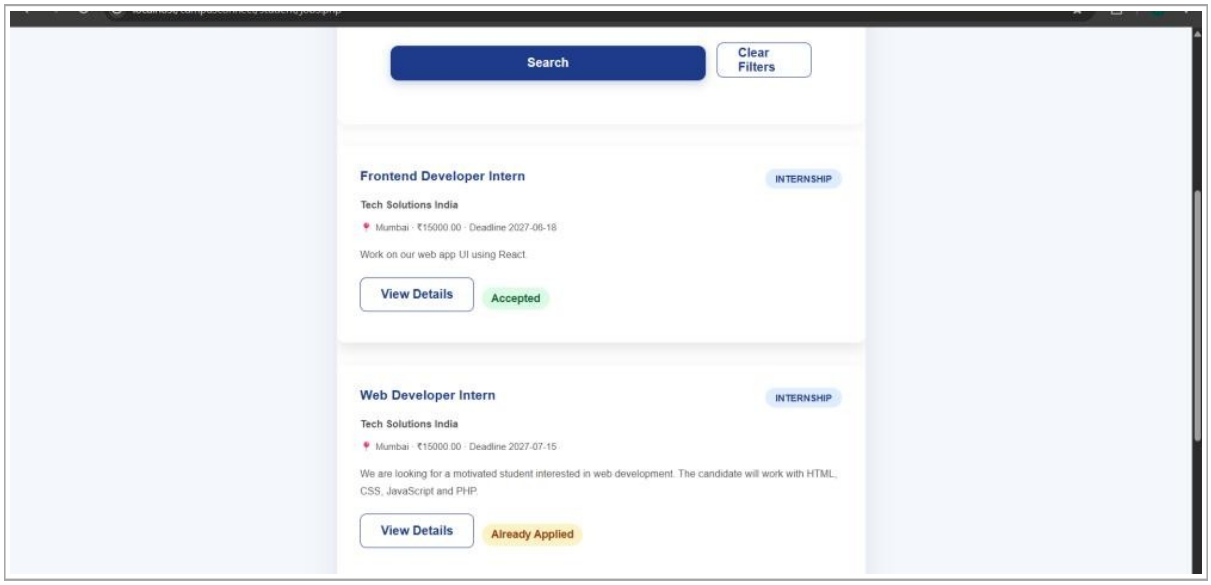


Figure 16: *Jobs listing – job cards with application status*

Opening a listing shows its full details, and applying presents a short cover-letter form. The Apply for Job page first summarises the job — title, company, location, job type, salary, application deadline and job description — so that the student can review it before applying.

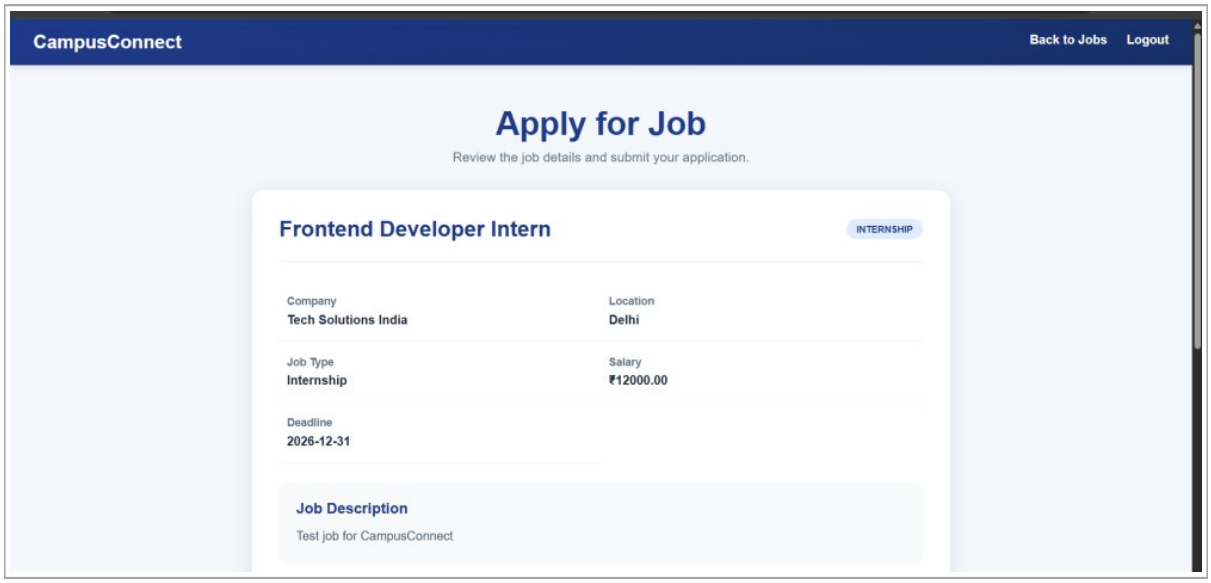


Figure 17: *Apply for Job – job summary*

Below the job summary, the Cover Letter section asks the student to explain why they are suitable for the position. The text area requires a minimum of 50 characters, and the application is submitted with the Submit Application button.

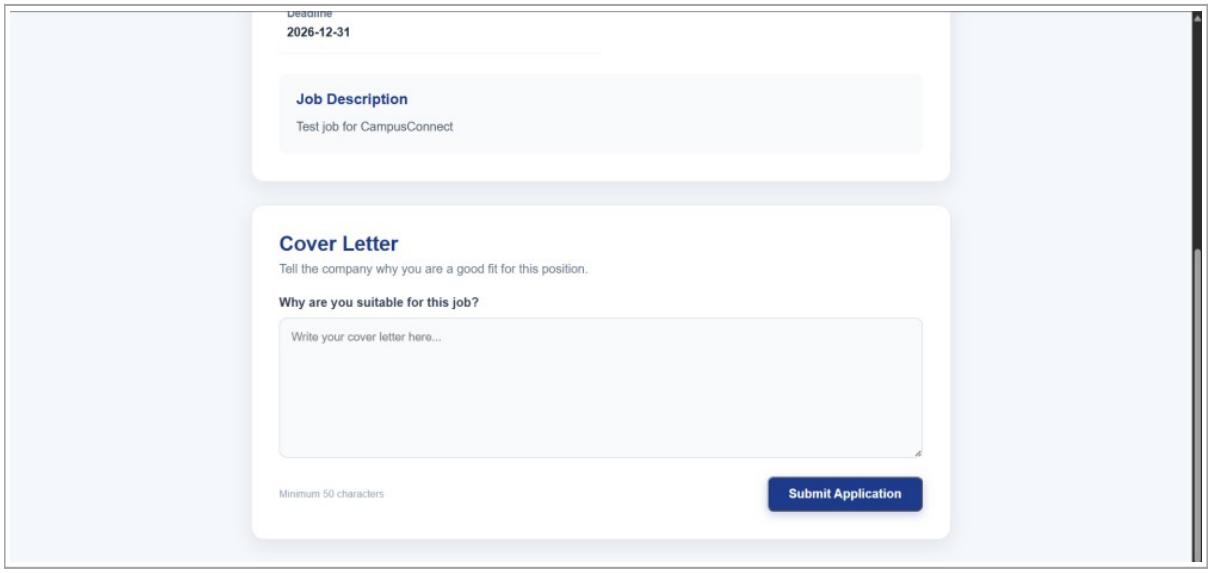


Figure 18: *Apply for Job – cover letter form*

The Job Details page shows the complete information of a listing — title, company, location, salary, application deadline and description. When the student has already applied, the application status badge (here Accepted) is displayed beside a Back to Jobs button instead of the apply form. The navigation bar gives access to the Dashboard, Jobs, My Applications, Profile and Logout.

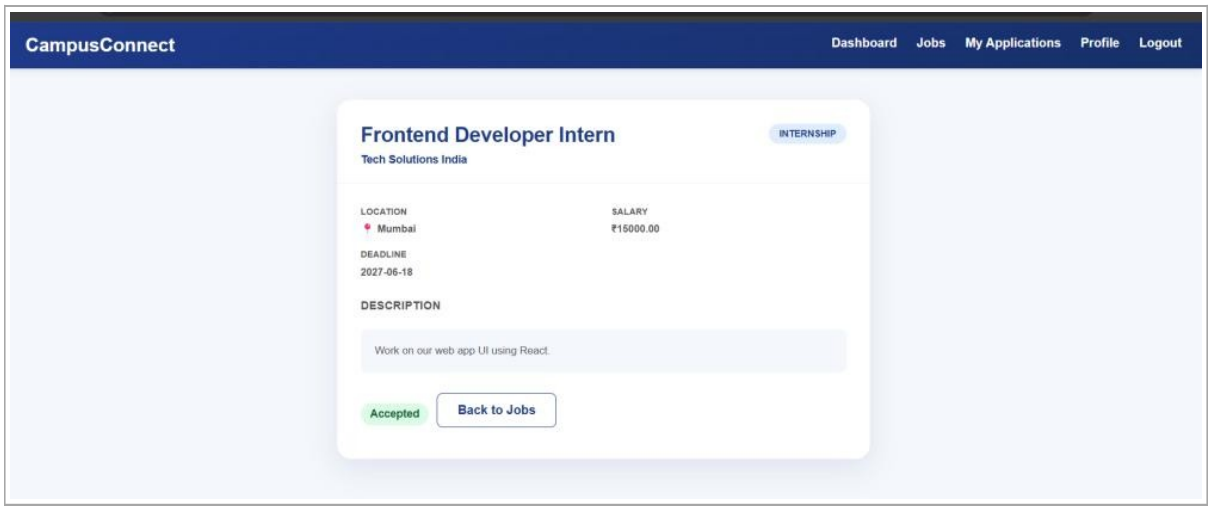


Figure 19: *Job Details page*

Every application the student submits can be reviewed later, along with its current status. The My Applications page lists each application as a card showing the job title, a job-type tag, a colour-coded status badge, the company, location, salary, application deadline, the date and time of application and the cover letter that was submitted. This example shows an application that the company has accepted.

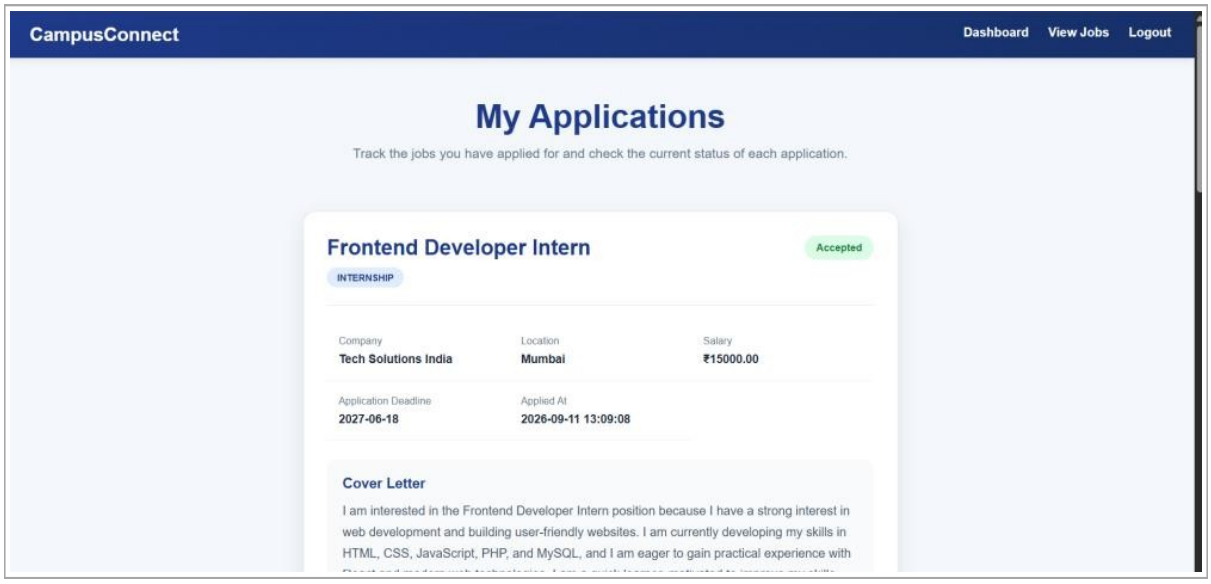


Figure 20: *My Applications – accepted application*

An application that the company has not yet acted on is shown in the same layout with an amber Pending badge. The card repeats the job details and the submitted cover letter, and the View Job button opens the full job details page for that listing.

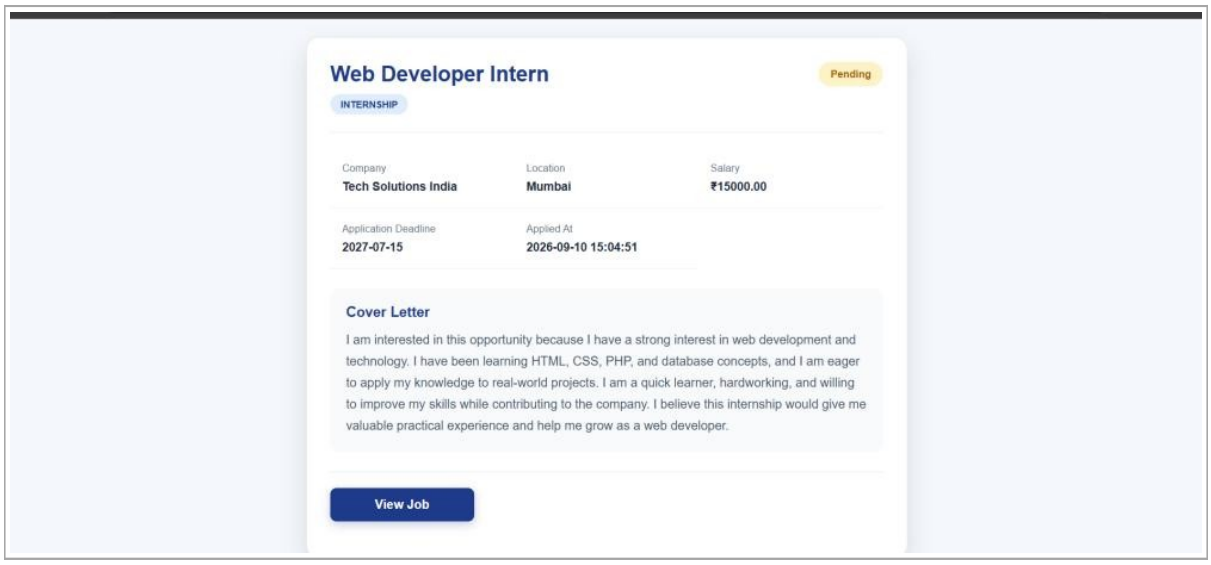


Figure 21: *My Applications – pending application*

Company Workflow

A company account has its own dashboard with quick links to every company-side action. The dashboard greets the company by name and presents four cards — Post a Job, View Applications, My Jobs and Company Profile — each with a short description and a link to the corresponding page.

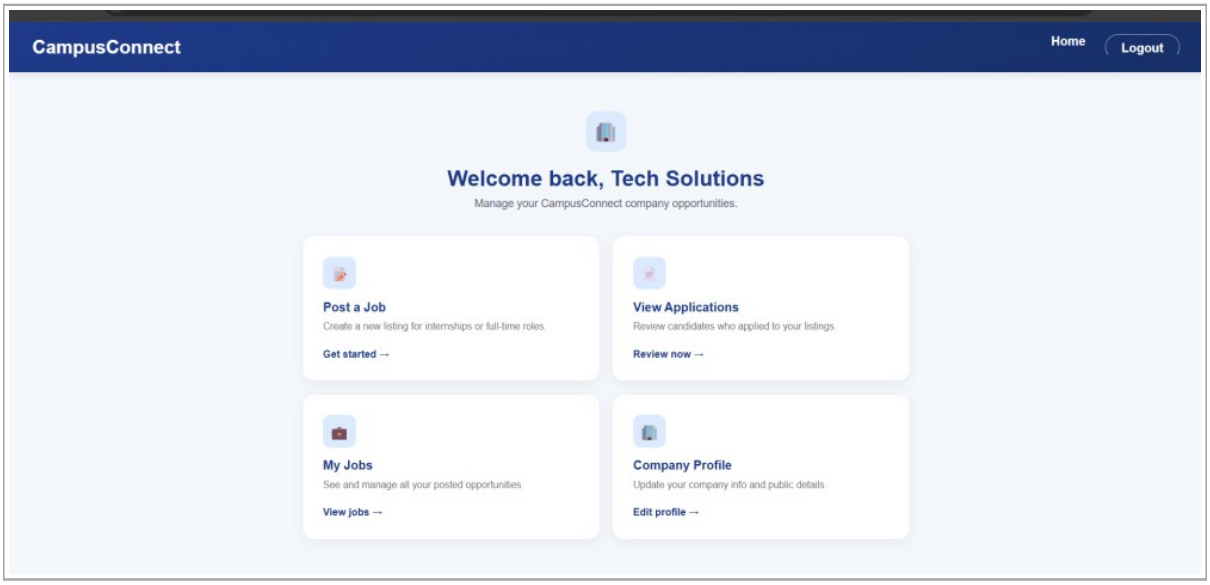


Figure 22: *Company Dashboard*

Companies post new job listings through a validated form. The Post a Job form uses browser validation, so leaving a required field blank, as shown here for the Job Title, displays a “Please fill out this field” message and prevents the job from being submitted until the field is completed.

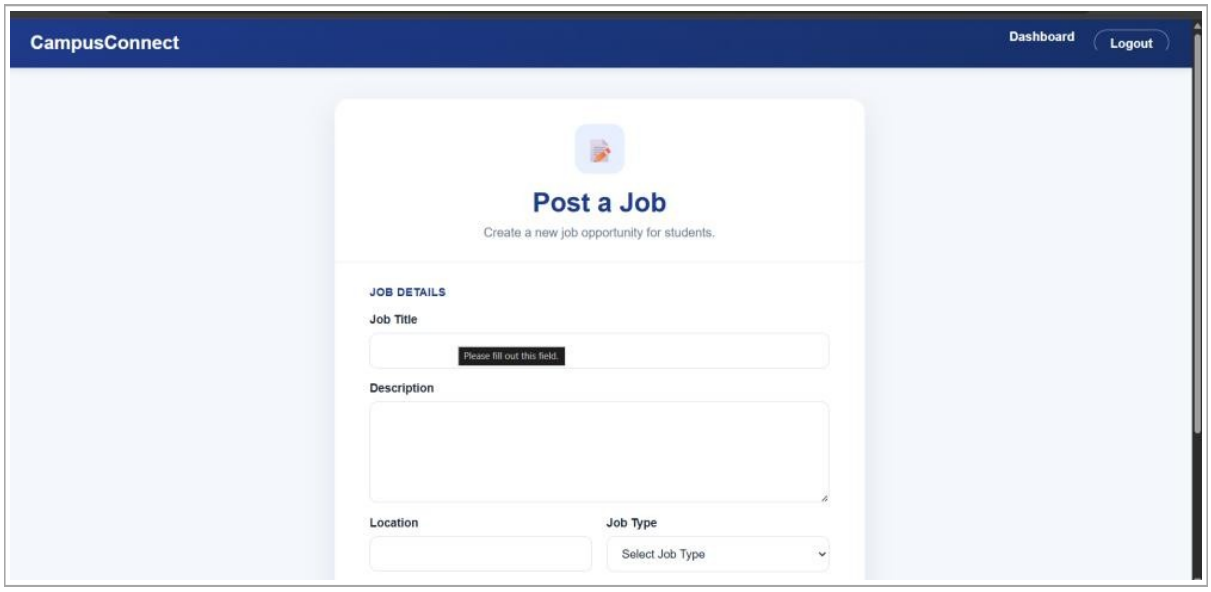


Figure 23: *Post a Job – form validation*

The complete Post a Job form groups its fields under two headings: Job Details, which holds the job title, description, location and job type, and Compensation & Timeline, which holds the salary and application deadline. The job is saved with the Post Job button.

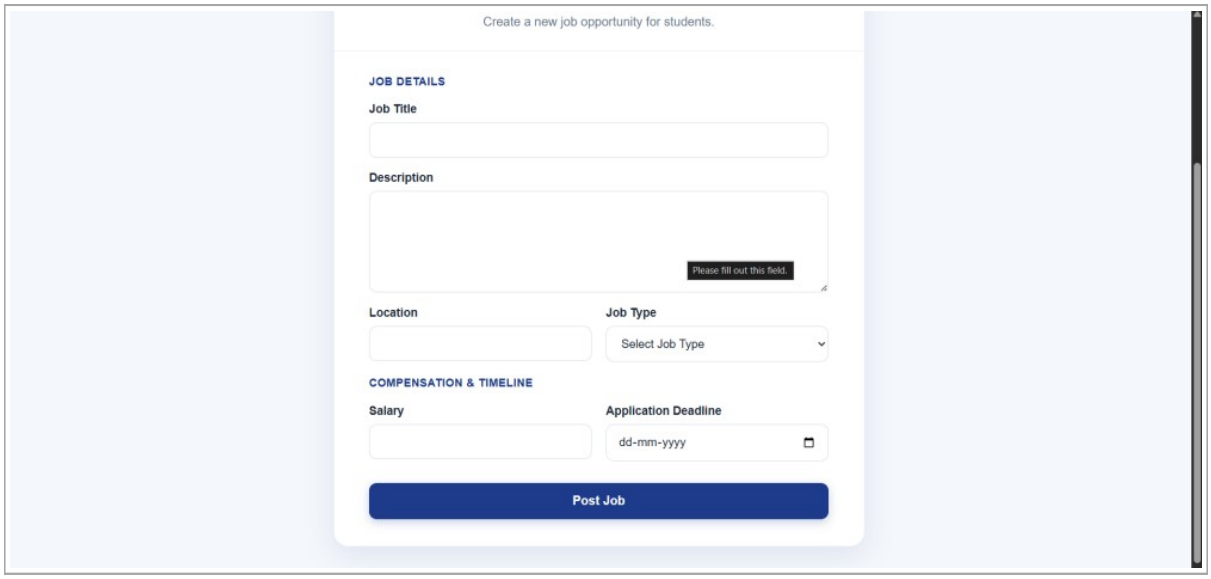


Figure 24: *Post a Job – full form*

Applications received against the company's own jobs can be filtered by status using the All, Pending, Accepted and Rejected tabs. Each application card shows the job title, a status badge, the applicant's name and email, a View resume link, the date of application, the cover letter and a View Student Profile button. For an application that has already been accepted, the company can also reset it back to pending.

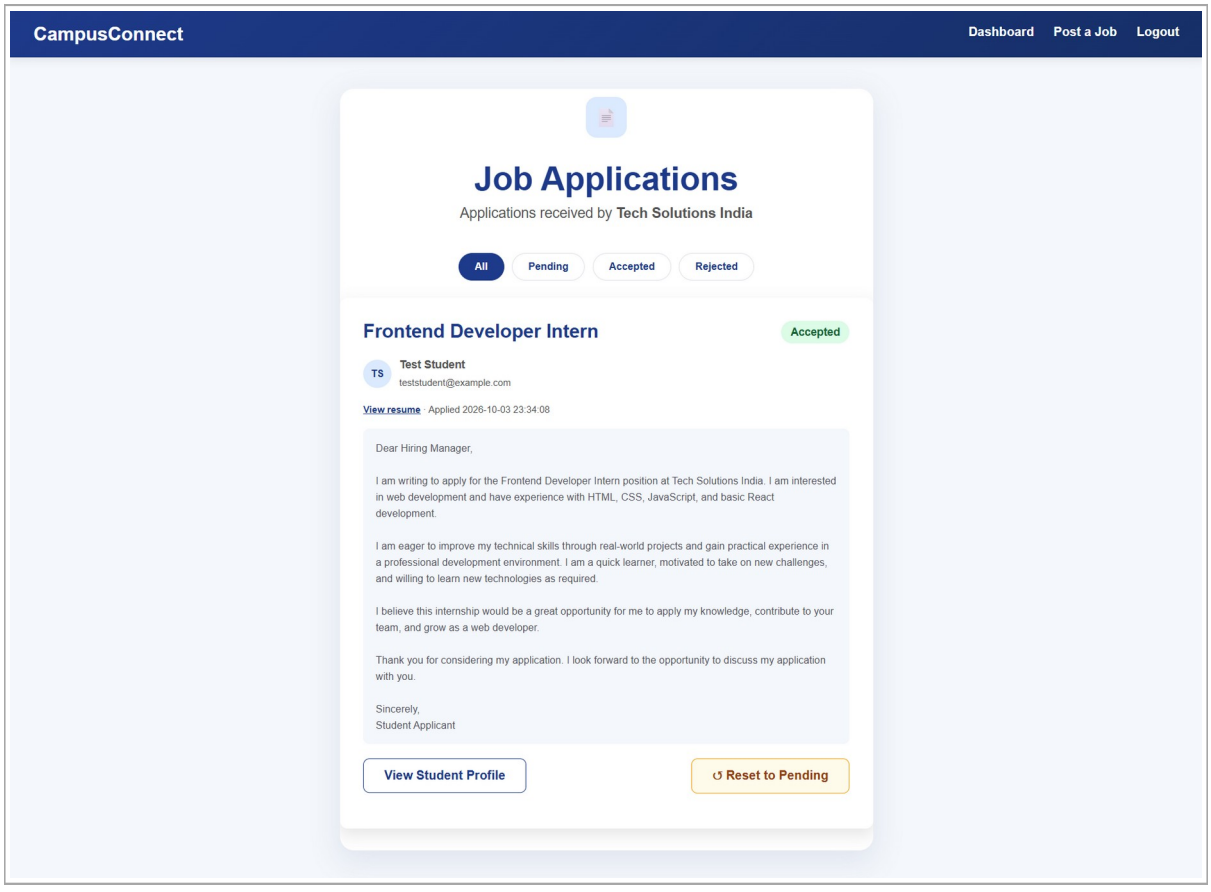


Figure 25: *Job Applications (company view) – filter tabs and accepted application*

Before taking a decision, the company can open the applicant's profile with the View Student Profile button on an application card. The page shows the applicant's academic information, skills and a link to the uploaded resume, so that the company can review the candidate in full before accepting or rejecting the application.

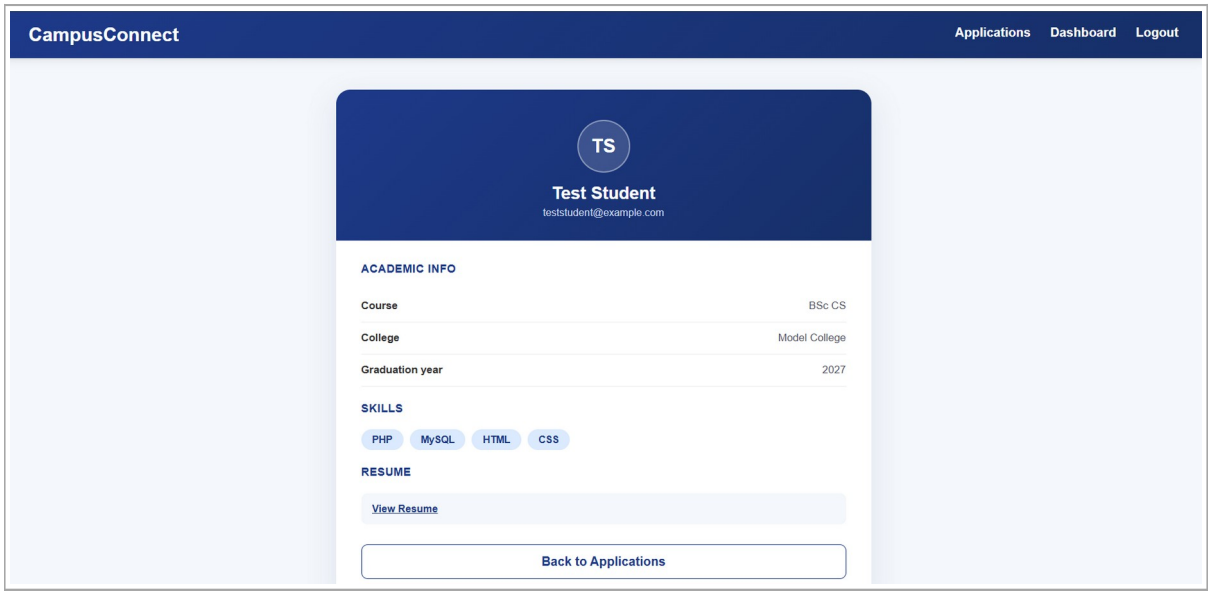


Figure 26: *Student profile as viewed by a company*

Pending applications can be accepted or rejected directly from the list. Each pending card shows the Accept and Reject buttons next to View Student Profile, and the chosen action updates the application status, which is then reflected on the student's My Applications page.

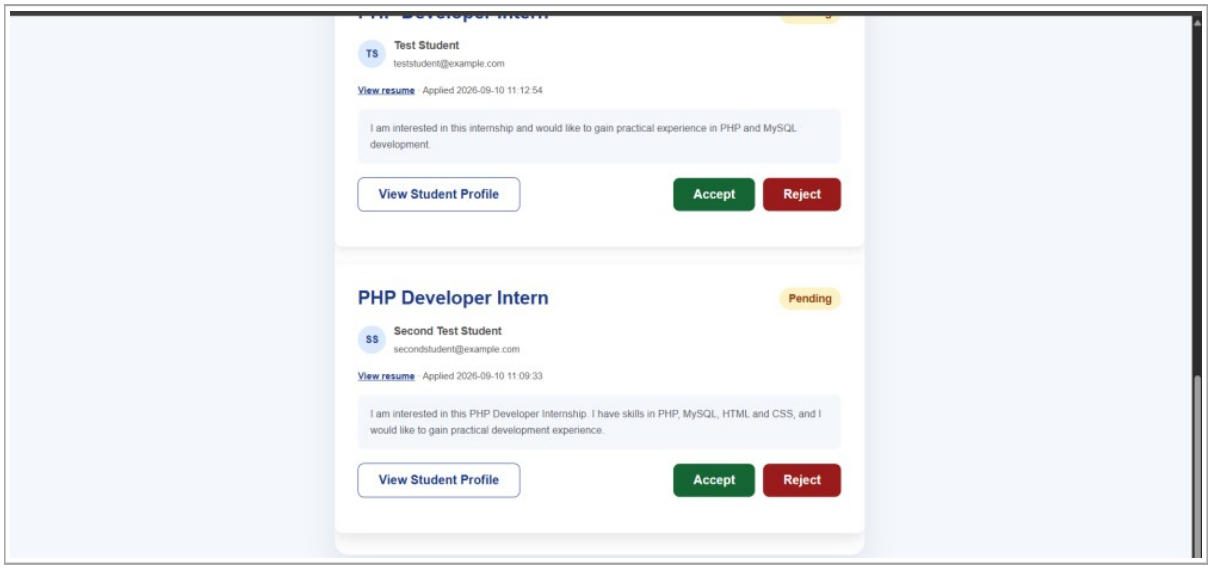


Figure 27: *Job Applications (company view) – pending applications with Accept / Reject*

Companies can also manage every job they have posted — editing, closing/reopening, or deleting a listing. The My Jobs page lists each job as a card with its status badge, job-type tag, description, location, job type, salary and application deadline, together with the Edit Job, Close Job and Delete Job buttons. A Post a New Job button is available at the top of the page.

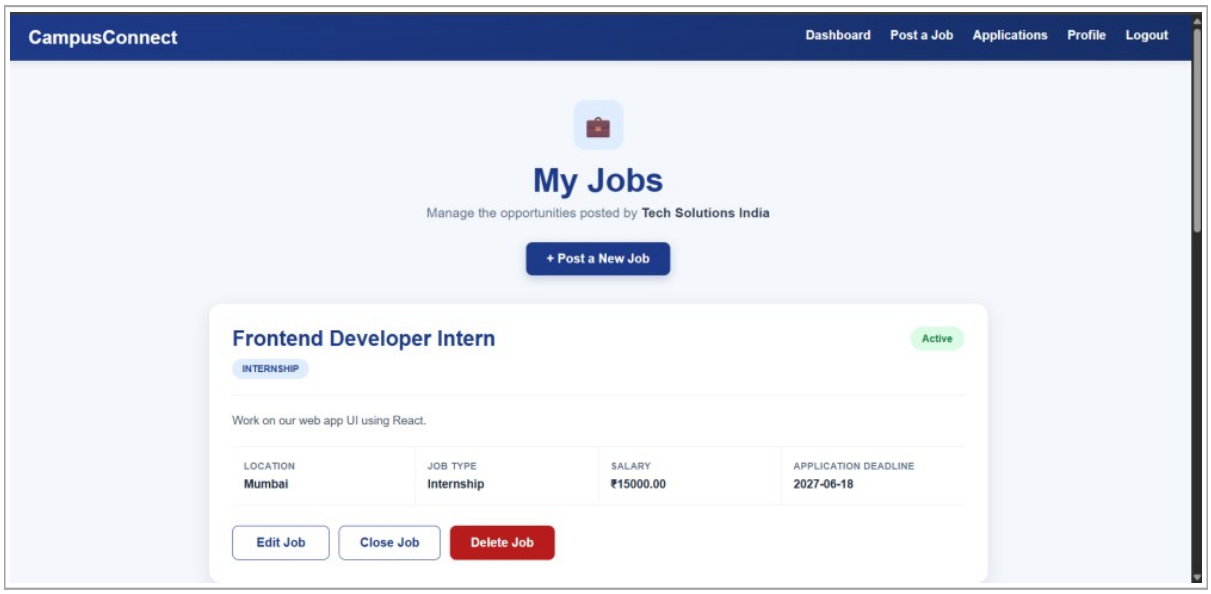


Figure 28: *My Jobs – posted job listing (1)*

The My Jobs page continues with the company's other postings. Every listing is presented with the same details and action buttons, so each job can be edited, closed or deleted individually.

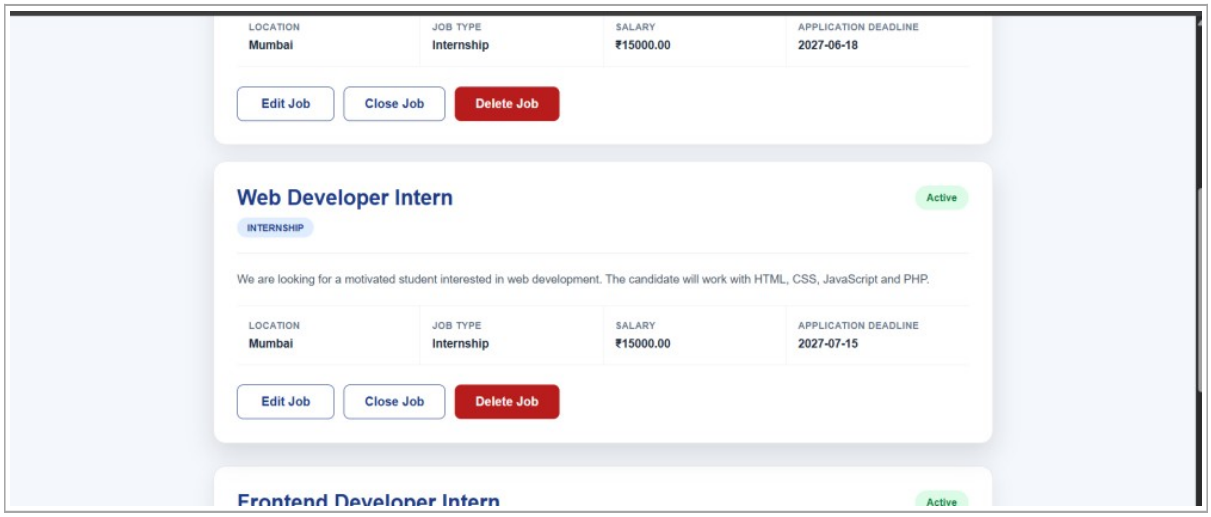


Figure 29: *My Jobs – posted job listing (2)*

The Edit Job form lets the company update the details of an existing listing. It opens pre-filled with the current job title, description, location and job type, followed by the compensation and timeline fields, so that only the details that need to change have to be edited.

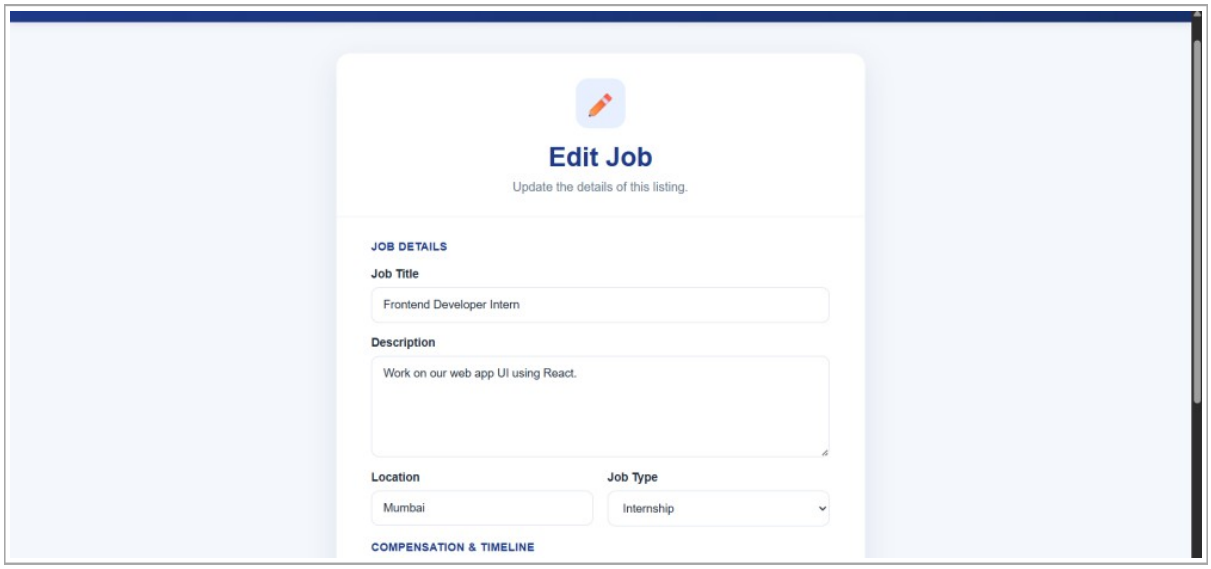


Figure 30: *Edit Job form*

Finally, a company can update its own public profile information. The Company Profile page displays the company name and company email in editable fields and saves any changes with the Update Profile button.

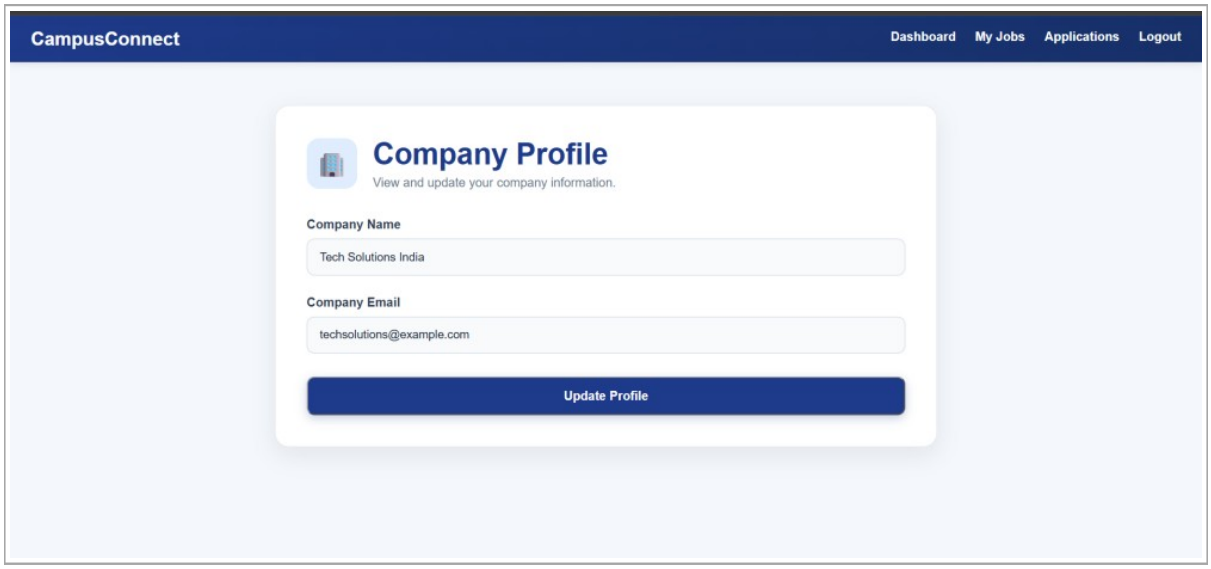


Figure 31: *Company Profile page*

Chapter 7: Conclusions

7.1 Conclusion

CampusConnect successfully delivers a working, two-role web platform that lets students discover and apply for jobs and internships, and lets companies post openings and manage the applications they receive. Every core feature identified in the requirements — registration, authentication, profile management, job posting and browsing, applying, and full application-status management — was implemented, tested, and confirmed working, as demonstrated by the test cases in Chapter 5 and the screenshots in Chapter 6.

7.1.1 Significance of the System

The system gives a small college a self-hosted, lightweight alternative to informal, manual placement coordination. Beyond its practical use, the project also demonstrates applied full-stack web development skills: relational database design with proper keys and constraints, server-side validation and session-based authentication, role-based access control, and a consistent, responsive user interface.

7.2 Limitations of the System

- The password-reset flow only verifies that the user knows the account's registered email address; it does not send a real, time-limited reset link by email, since that would require configuring external SMTP infrastructure not available in this local academic setup.
- There is no dedicated administrator panel to moderate job postings or manage user accounts directly at the database level.
- Resumes are stored on the local server's filesystem rather than a dedicated cloud storage service.
- The internships database table exists but is not yet wired into any part of the application (see Section 4.2.1).
- The system has been tested locally on XAMPP and has not yet been deployed to a public web server.

7.3 Future Scope of the Project

- Build out the internships module properly using the existing internships table — letting a company convert an accepted application into a tracked internship with start/end dates, stipend and a completion certificate.
- Add a real, email-based password-reset flow using an SMTP service, sending a secure, time-limited reset link.
- Add an administrator dashboard to moderate job postings and manage user accounts.
- Add resume preview or basic keyword-based resume/skill matching to help companies shortlist applicants faster.
- Deploy the system to a public server with HTTPS so it can be used beyond a local XAMPP environment.

References

[1] PHP Documentation. https://www.php.net/docs.php

[2] MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/

[3] Mozilla Developer Network. HTML and CSS Reference. https://developer.mozilla.org/

[4] W3Schools. PHP and MySQL Tutorials. https://www.w3schools.com/php/

[5] Apache Friends. XAMPP Documentation. https://www.apachefriends.org/

Glossary

Term	Meaning
PHP	Hypertext Preprocessor — the server-side scripting language used to build CampusConnect's back end.
SQL	Structured Query Language — the language used to define and query the MySQL database.
XAMPP	A free, local server package bundling Apache, MySQL/MariaDB, PHP and phpMyAdmin.
CRUD	Create, Read, Update, Delete — the four basic database operations used throughout the application.
DFD	Data Flow Diagram — a diagram showing how data moves between external entities, processes and data stores.
ERD	Entity-Relationship Diagram — a diagram showing database tables and the relationships between them.
Session	A server-side mechanism (PHP's $_SESSION) used to remember a logged-in user across multiple page requests.
Prepared Statement	A parameterised SQL query, prepared once and executed with bound values, used to prevent SQL injection.
Foreign Key	A column that references the primary key of another table, enforcing a relationship between the two.

Appendix A: Source Code Listings

The full source code of every page is available in the project directory (C:\xampp\htdocs\campusconnect\). The key source files are listed in this appendix. Each listing is given on a separate page, together with a one-line statement of its purpose.

Table A.1: Source files included in this appendix

No.	File	Purpose
1	login.php	Authenticates a user with email and password and redirects to the student or company dashboard.
2	company/applications.php	Lists the applications received by a company, filtered by status, with the Accept, Reject and Reset actions.
3	student/register.php	Registers a new student account together with the student's academic profile.
4	company/post_job.php	Lets a company post a new job or internship listing.
5	company/register.php	Registers a new company account together with the company profile.
6	company/dashboard.php	Company dashboard presenting four action cards: Post a Job, View Applications, My Jobs and Company Profile.
7	company/edit_job.php	Lets a company edit a job it has posted, after checking that the job belongs to that company.
8	company/student_profile.php	Read-only view of a student's profile, shown to a company before an accept or reject decision.
9	student/edit_profile.php	Lets a student update academic details and upload a resume (PDF, DOC or DOCX, up to 5 MB).
10	student/job_details.php	Shows the full details of a job and the student's current application status, preventing duplicate applications.
11	forgot_password.php	First step of password reset: checks that the entered email belongs to a registered account.
12	reset_password.php	Second step of password reset: stores a new hashed password for the verified account.

login.php

Purpose: Authenticates a user with email and password and redirects to the student or company dashboard.

```
<?php
session_start();
require_once "config/database.php";

$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $email = trim($_POST["email"]);
    $password = $_POST["password"];

    $stmt = $conn->prepare(
        "SELECT id, name, password, role, status
         FROM users
         WHERE email = ?"
    );
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows === 1) {
        $user = $result->fetch_assoc();

        if ($user["status"] !== "active") {
            $message = "Your account is not active.";
        } elseif (password_verify($password, $user["password"])) {
            $_SESSION["user_id"] = $user["id"];
            $_SESSION["name"] = $user["name"];
            $_SESSION["role"] = $user["role"];

            if ($user["role"] === "student") {
                header("Location: student/dashboard.php");
                exit;
            }

            if ($user["role"] === "company") {
                header("Location: company/dashboard.php");
                exit;
            }
        } else {
            $message = "Invalid email or password.";
        }
    } else {
        $message = "Invalid email or password.";
    }

    $stmt->close();
}
?>
```

company/applications.php

Purpose: Lists the applications received by a company, filtered by status, with the Accept, Reject and Reset actions.

```
<?php
session_start();
require_once "../config/database.php";

if (!isset($_SESSION["user_id"]) || $_SESSION["role"] !== "company") {
    header("Location: ../login.php");
    exit;
}

$user_id = $_SESSION["user_id"];
$filter = $_GET["status"] ?? "all";

if ($filter !== "all" && $filter !== "pending" &&
    $filter !== "accepted" && $filter !== "rejected") {
    $filter = "all";
}

$company_stmt = $conn->prepare(
    "SELECT companies.id, companies.name, companies.email
     FROM users INNER JOIN companies ON users.email = companies.email
     WHERE users.id = ?"
);
$company_stmt->bind_param("i", $user_id);
$company_stmt->execute();
$company_result = $company_stmt->get_result();
$company = $company_result->fetch_assoc();
$company_stmt->close();

if (!$company) { die("Company profile not found."); }
$company_id = $company["id"];

$applications_stmt = $conn->prepare(
    "SELECT applications.id, applications.cover_letter,
            applications.applied_at, applications.status,
            jobs.title AS job_title,
            users.name AS student_name, users.email AS student_email,
            students.id AS student_id, students.resume AS student_resume
     FROM applications
     INNER JOIN jobs ON applications.job_id = jobs.id
     INNER JOIN students ON applications.student_id = students.id
     INNER JOIN users ON students.user_id = users.id
     WHERE jobs.company_id = ?
       AND (? = 'all' OR applications.status = ?)
     ORDER BY applications.applied_at DESC"
);
$applications_stmt->bind_param("iss", $company_id, $filter, $filter);
$applications_stmt->execute();
$applications_result = $applications_stmt->get_result();
?>
```

student/register.php

Purpose: Registers a new student account together with the student's academic profile.

```
<?php
require_once "../config/database.php";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST["name"]);
    $email = trim($_POST["email"]);
    $password = $_POST["password"];
    $confirm_password = $_POST["confirm_password"];

    if ($password !== $confirm_password) {
        $message = "Passwords do not match.";
    } else {
        $course = trim($_POST["course"]);
        $college = trim($_POST["college"]);
        $graduation_year = $_POST["graduation_year"];
        $skills = trim($_POST["skills"]);

        $hashed_password = password_hash($password, PASSWORD_DEFAULT);
        $role = "student";
        $phone = "";
        $status = "active";

        $stmt = $conn->prepare(
            "INSERT INTO users (name, email, password, role, phone, status)
             VALUES (?, ?, ?, ?, ?, ?)"
        );
        $stmt->bind_param("ssssss", $name, $email, $hashed_password,
                          $role, $phone, $status);

        if ($stmt->execute()) {
            $user_id = $conn->insert_id;

            $student_stmt = $conn->prepare(
                "INSERT INTO students
                 (user_id, course, college, graduation_year, resume, skills)
                 VALUES (?, ?, ?, ?, ?, ?)"
            );
            $resume = "";
            $student_stmt->bind_param("ississ", $user_id, $course,
                $college, $graduation_year, $resume, $skills);

            if ($student_stmt->execute()) {
                echo "Student account and profile created successfully!";
            } else {
                echo "Student profile error: " . $student_stmt->error;
            }
            $student_stmt->close();
        } else {
            echo "Account error: " . $stmt->error;
        }
        $stmt->close();
    }
}
?>
```

company/post_job.php

Purpose: Lets a company post a new job or internship listing.

```
<?php
session_start();
require_once "../config/database.php";

if (!isset($_SESSION["user_id"]) || $_SESSION["role"] !== "company") {
    header("Location: ../login.php");
    exit;
}

$user_id = $_SESSION["user_id"];
$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $user_stmt = $conn->prepare("SELECT email FROM users WHERE id = ?");
    $user_stmt->bind_param("i", $user_id);
    $user_stmt->execute();
    $user = $user_stmt->get_result()->fetch_assoc();
    $user_stmt->close();

    $company_stmt = $conn->prepare("SELECT id FROM companies WHERE email = ?");
    $company_stmt->bind_param("s", $user["email"]);
    $company_stmt->execute();
    $company = $company_stmt->get_result()->fetch_assoc();
    $company_stmt->close();

    if (!$company) { die("Company profile not found."); }
    $company_id = $company["id"];

    $title = trim($_POST["title"]);
    $description = trim($_POST["description"]);
    $location = trim($_POST["location"]);
    $job_type = trim($_POST["job_type"]);
    $salary = $_POST["salary"];
    $deadline = $_POST["deadline"];
    $status = "active";

    $stmt = $conn->prepare(
        "INSERT INTO jobs (company_id, title, description, location,
                           job_type, salary, deadline, status)
         VALUES (?, ?, ?, ?, ?, ?, ?, ?)"
    );
    $stmt->bind_param("issssdss", $company_id, $title, $description,
                      $location, $job_type, $salary, $deadline, $status);

    if ($stmt->execute()) { $message = "Job posted successfully!"; }
    else { $message = "Error: " . $stmt->error; }
    $stmt->close();
}
?>
```

company/register.php

Purpose: Registers a new company account together with the company profile.

```
<?php
require_once "../config/database.php";

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $company_name = trim($_POST["company_name"]);
    $email = trim($_POST["email"]);
    $password = $_POST["password"];
    $confirm_password = $_POST["confirm_password"];

    if ($password !== $confirm_password) {
        $message = "Passwords do not match.";
    } else {

        $hashed_password = password_hash($password, PASSWORD_DEFAULT);
        $role = "company";
        $phone = "";
        $status = "active";

        $stmt = $conn->prepare(
            "INSERT INTO users (name, email, password, role, phone, status)
             VALUES (?, ?, ?, ?, ?, ?)"
        );
        $stmt->bind_param(
            "ssssss",
            $company_name, $email, $hashed_password, $role, $phone, $status
        );

        if ($stmt->execute()) {

            $company_stmt = $conn->prepare(
                "INSERT INTO companies
                (name, email, phone, website, industry, description)
                VALUES (?, ?, ?, ?, ?, ?)"
            );
            $website = "";
            $industry = "";
            $description = "";

            $company_stmt->bind_param(
                "ssssss",
                $company_name, $email, $phone, $website, $industry, $description
            );
            $company_stmt->execute();
            $company_stmt->close();

        } else {
            echo "Account error: " . $stmt->error;
        }

        $stmt->close();
    }
}
?>
```

company/dashboard.php

Purpose: Company dashboard presenting four action cards: Post a Job, View Applications, My Jobs and Company Profile.

```
<?php
session_start();

if (!isset($_SESSION["user_id"]) || $_SESSION["role"] !== "company") {
    header("Location: ../login.php");
    exit;
}

$name = $_SESSION["name"];
?>
<!-- Dashboard presents four action cards: Post a Job, View      -->
<!-- Applications, My Jobs, Company Profile - each linking to    -->
<!-- its respective page. -->

<div class="action-grid">
    <a href="post_job.php" class="action-card">
        <h2>Post a Job</h2>
        <p>Create a new listing for internships or full-time roles.</p>
    </a>

    <a href="applications.php" class="action-card">
        <h2>View Applications</h2>
        <p>Review candidates who applied to your listings.</p>
    </a>

    <a href="my_jobs.php" class="action-card">
        <h2>My Jobs</h2>
        <p>See and manage all your posted opportunities.</p>
    </a>

    <a href="profile.php" class="action-card">
        <h2>Company Profile</h2>
        <p>Update your company info and public details.</p>
    </a>
</div>
```

company/edit_job.php

Purpose: Lets a company edit a job it has posted, after checking that the job belongs to that company.

```
<?php
session_start();

require_once "../config/database.php";

if (!isset($_SESSION["user_id"]) || $_SESSION["role"] !== "company") {
    header("Location: ../login.php");
    exit;
}

if (!isset($_GET["job_id"]) || !is_numeric($_GET["job_id"])) {
    die("Invalid job.");
}
$job_id = (int) $_GET["job_id"];
$user_id = $_SESSION["user_id"];
$company_stmt = $conn->prepare(
    "SELECT companies.id FROM users
     INNER JOIN companies ON users.email = companies.email
     WHERE users.id = ?"
);
$company_stmt->bind_param("i", $user_id);
$company_stmt->execute();
$company = $company_stmt->get_result()->fetch_assoc();
$company_stmt->close();
if (!$company) { die("Company profile not found."); }
$company_id = $company["id"];
$job_stmt = $conn->prepare(
    "SELECT id, title, description, location, job_type,
            salary, deadline, status
     FROM jobs WHERE id = ? AND company_id = ?"
);
$job_stmt->bind_param("ii", $job_id, $company_id);
$job_stmt->execute();
$job = $job_stmt->get_result()->fetch_assoc();
$job_stmt->close();
if (!$job) { die("Job not found."); }
$message = "";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $title = trim($_POST["title"] ?? "");
    $description = trim($_POST["description"] ?? "");
    $location = trim($_POST["location"] ?? "");
    $job_type = trim($_POST["job_type"] ?? "");
    $salary = trim($_POST["salary"] ?? "");
    $deadline = trim($_POST["deadline"] ?? "");
    $status = trim($_POST["status"] ?? "");
    $update_stmt = $conn->prepare(
        "UPDATE jobs
         SET title=?, description=?, location=?, job_type=?,
             salary=?, deadline=?, status=?
         WHERE id=? AND company_id=?"
    );
    $update_stmt->bind_param(
        "sssssssii",
        $title, $description, $location, $job_type,
        $salary, $deadline, $status, $job_id, $company_id
    );
    if ($update_stmt->execute()) {
        $message = "Job updated successfully.";
    } else {
        $message = "Update error: " . $update_stmt->error;
    }
    $update_stmt->close();
}
?>
```

company/student_profile.php

Purpose: Read-only view of a student's profile, shown to a company before an accept or reject decision.

```
<?php
session_start();
require_once "../config/database.php";

if (!isset($_SESSION["user_id"]) || $_SESSION["role"] !== "company") {
    header("Location: ../login.php");
    exit;
}

$student_id = (int) $_GET["student_id"];

$stmt = $conn->prepare(
    "SELECT users.name, users.email, students.course,
            students.college, students.graduation_year,
            students.resume, students.skills
     FROM students
     INNER JOIN users ON students.user_id = users.id
     WHERE students.id = ?"
);
$stmt->bind_param("i", $student_id);
$stmt->execute();
$student = $stmt->get_result()->fetch_assoc();
$stmt->close();

if (!$student) {
    die("Student profile not found.");
}
?>
<!-- Read-only view of a student's profile, shown to a company    -->
<!-- from the Job Applications page before an Accept/Reject       -->
<!-- decision is made. -->
```

student/edit_profile.php

Purpose: Lets a student update academic details and upload a resume (PDF, DOC or DOCX, up to 5 MB).

```
<?php
session_start();
require_once "../config/database.php";

if (!isset($_SESSION["user_id"]) || $_SESSION["role"] !== "student") {
    header("Location: ../login.php");
    exit;
}

$user_id = $_SESSION["user_id"];

$stmt = $conn->prepare(
    "SELECT course, college, graduation_year, skills, resume
     FROM students WHERE user_id = ?"
);
$stmt->bind_param("i", $user_id);
$stmt->execute();
$student = $stmt->get_result()->fetch_assoc();
$stmt->close();

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $resume = $student["resume"] ?? "";
    $resumeValid = true;

    $course = trim($_POST["course"]);
    $college = trim($_POST["college"]);
    $graduation_year = trim($_POST["graduation_year"]);
    $skills = trim($_POST["skills"]);

    if (isset($_FILES["resume"]) && $_FILES["resume"]["error"] === 0) {

        if ($_FILES["resume"]["size"] > 5 * 1024 * 1024) {
            $message = "Resume must be smaller than 5 MB.";
            $resumeValid = false;
        } else {

            $resume = uniqid("resume_", true) . "." .
                strtolower(pathinfo($_FILES["resume"]["name"], PATHINFO_EXTENSION));

            $allowed = ["pdf", "doc", "docx"];
            $ext = strtolower(pathinfo($resume, PATHINFO_EXTENSION));

            if (!in_array($ext, $allowed)) {
                $message = "Only PDF, DOC, and DOCX files are allowed.";
                $resumeValid = false;
            } else {
                move_uploaded_file(
                    $_FILES["resume"]["tmp_name"],
                    "../resume_uploads/" . $resume
                );
            }
        }
    }

    if ($resumeValid) {
        $stmt = $conn->prepare(
            "UPDATE students
             SET course=?, college=?, graduation_year=?, skills=?, resume=?
             WHERE user_id=?"
        );
        $stmt->bind_param(
            "ssissi",
            $course, $college, $graduation_year, $skills, $resume, $user_id
        );
        $stmt->execute();
        $stmt->close();
    }
}
?>
```

student/job_details.php

Purpose: Shows the full details of a job and the student's current application status, preventing duplicate applications.

```
<?php
session_start();

require_once "../config/database.php";

if (!isset($_SESSION["user_id"]) || $_SESSION["role"] !== "student") {
    header("Location: ../login.php");
    exit;
}

if (!isset($_GET["job_id"]) || !is_numeric($_GET["job_id"])) {
    die("Invalid job.");
}

$job_id = (int) $_GET["job_id"];

$stmt = $conn->prepare(
    "SELECT
        jobs.id,
        jobs.title,
        jobs.description,
        jobs.location,
        jobs.job_type,
        jobs.salary,
        jobs.deadline,
        companies.name AS company_name
     FROM jobs
     INNER JOIN companies
        ON jobs.company_id = companies.id
     WHERE jobs.id = ?"
);

$stmt->bind_param("i", $job_id);
$stmt->execute();

$result = $stmt->get_result();
$job = $result->fetch_assoc();

$stmt->close();

if (!$job) {
    die("Job not found or no longer available.");
}
$user_id = $_SESSION["user_id"];

$student_stmt = $conn->prepare(
    "SELECT id
     FROM students
     WHERE user_id = ?"
);

$student_stmt->bind_param("i", $user_id);
$student_stmt->execute();

$student_result = $student_stmt->get_result();
$student = $student_result->fetch_assoc();

$student_stmt->close();

if (!$student) {
    die("Student profile not found.");
}

$student_id = $student["id"];
```

```
$application_stmt = $conn->prepare(
    "SELECT status
     FROM applications
     WHERE student_id = ? AND job_id = ?"
);

$application_stmt->bind_param(
    "ii",
    $student_id,
    $job_id
);

$application_stmt->execute();

$application_result = $application_stmt->get_result();
$application = $application_result->fetch_assoc();

$application_stmt->close();
?>
<!-- If $application exists, the page shows the current status    -->
<!-- (Accepted / Rejected / Already Applied) instead of an Apply   -->
<!-- Now button, preventing duplicate applications. -->
```

forgot_password.php

Purpose: First step of password reset: checks that the entered email belongs to a registered account.

```
<?php
session_start();
require_once "config/database.php";

$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $email = trim($_POST["email"]);

    $stmt = $conn->prepare("SELECT id FROM users WHERE email = ?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows === 1) {
        $_SESSION["reset_email"] = $email;
        header("Location: reset_password.php");
        exit;
    } else {
        $message = "No account found with that email address.";
    }

    $stmt->close();
}
?>
```

reset_password.php

Purpose: Second step of password reset: stores a new hashed password for the verified account.

```
<?php
session_start();
require_once "config/database.php";

if (!isset($_SESSION["reset_email"])) {
    header("Location: forgot_password.php");
    exit;
}

$email = $_SESSION["reset_email"];
$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $password = $_POST["password"];
    $confirm_password = $_POST["confirm_password"];

    if ($password !== $confirm_password) {
        $message = "Passwords do not match.";
    } else {

        $hashed_password = password_hash($password, PASSWORD_DEFAULT);

        $stmt = $conn->prepare("UPDATE users SET password = ? WHERE email = ?");
        $stmt->bind_param("ss", $hashed_password, $email);

        if ($stmt->execute()) {
            unset($_SESSION["reset_email"]);
            header("Location: login.php");
            exit;
        } else {
            $message = "Something went wrong. Please try again.";
        }

        $stmt->close();
    }
}
?>
```

Appendix B: Installation and Setup Guide

The steps below explain how to run CampusConnect locally, matching the environment used throughout development.

1. Install XAMPP and start the Apache and MySQL/MariaDB modules from the XAMPP Control Panel.
2. Copy the project folder into C:\xampp\htdocs\campusconnect\.
3. Open phpMyAdmin (http://localhost/phpmyadmin), create a database named campusconnect_db, and import the project's schema (tables: users, students, companies, jobs, applications).
4. Open a browser and navigate to http://localhost/campusconnect/index.php to load the home page.
5. Register a student account and a company account to explore both roles, or use existing test accounts if provided separately.